

Lab1

一. Lab1流程

0. 示例代码获取及说明

0.1 后端代码

后端代码使用SpringBoot框架，数据库有jpa版本和mybatis版本，自行选择

jpa版本

命令行输入

```
git clone http://{注册门户使用的邮箱}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todolist-backend.git
```

或

```
git clone http://{注册门户的用户名}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todolist-backend.git
```

例如：

邮箱中的 @ 要转码写成 %40

```
git clone http://123123123%40.com:321321321@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todolist-backend.git
```

或

```
git clone http://yourname:yourpswd@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todolist-backend.git
```

mybatis版本

命令行输入

```
git clone http://{注册门户使用的邮箱}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git
```

或

```
git clone http://{注册门户的用户名}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git
```

例如：

邮箱中的 @ 要转码写成 %40

```
git clone http://123123123%40.com:321321321@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git
```

或

git clone <http://yourname:yourpswd@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git>

0.2 前端代码

前端代码使用vue框架

命令行输入

git clone [http://{注册门户使用的邮箱}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git](http://注册门户使用的邮箱:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git)

或

git clone <http://{注册门户的用户名}:门户密码@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git>

例如：

邮箱中的 @ 要转码写成 %40

git clone <http://123123123%40.com:321321321@http://seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git>

或

git clone <http://yourname:yourpswd@http://seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git>

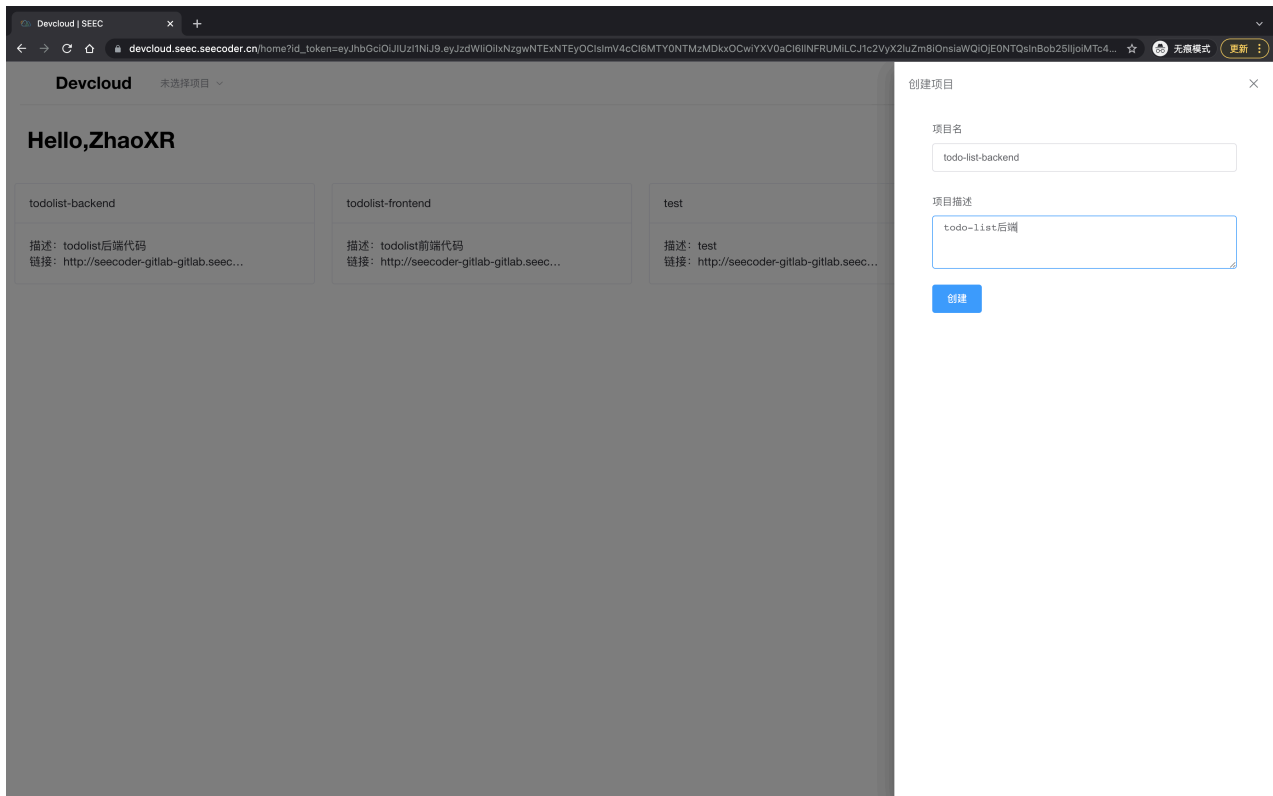
1. 项目初始化

1.1 创建项目

只需要小组内一名同学创建项目，其他成员通过邀请的形式加入项目中，无需额外创建项目

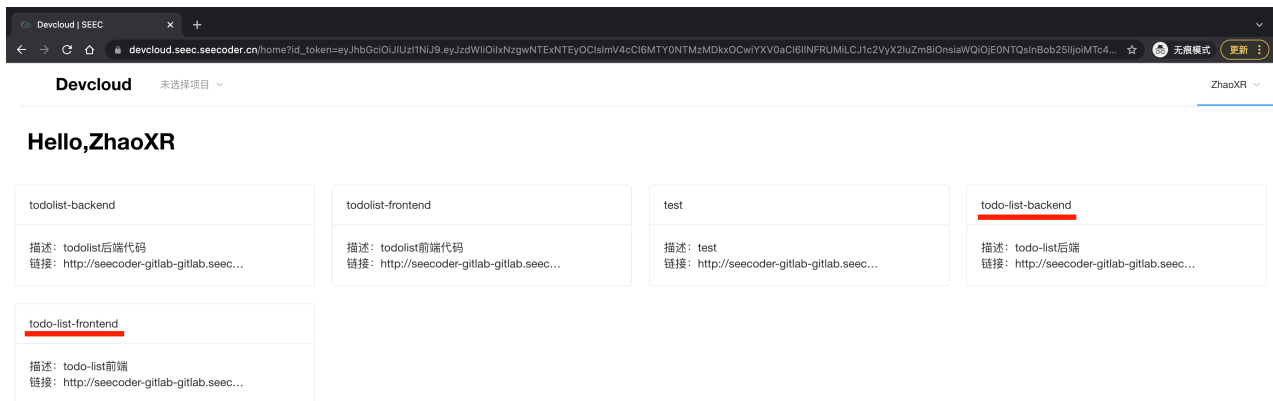
点击**右上角id**右边的下拉框，选择创建项目

输入项目名称，项目名称只能为以小写英文字母开头，且由英文字母和‘-’组成的字符串，长度为3-30



为前后端项目各创建一个项目

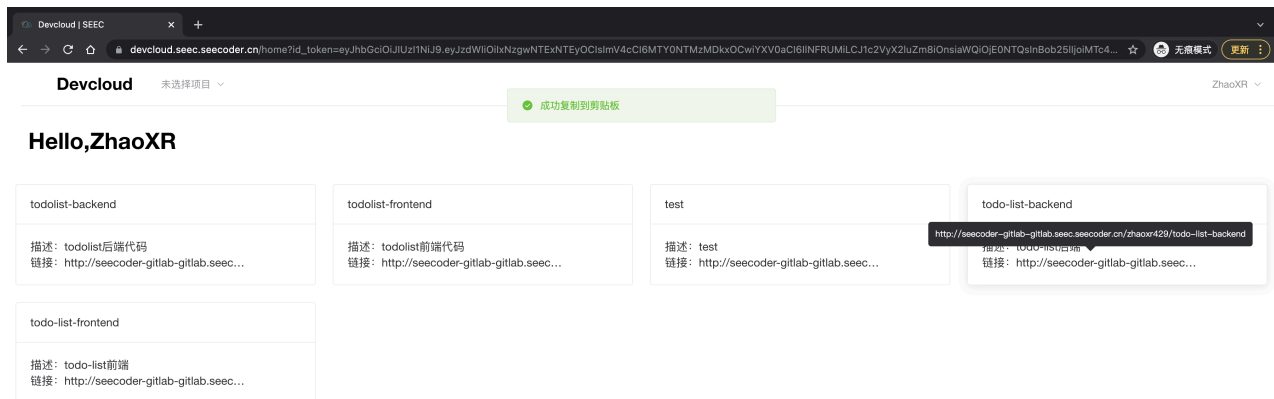
创建完成如下图，有 `todo-list-backend` 以及 `todo-list-frontend` 一个后端、一个前端项目



1.2 上传项目

使用 `Git`，通过自动创建的仓库地址，将项目clone到本地

在首页点击项目链接即可复制，加上 `.git` 后缀，加上注册门户时的邮箱/用户名和密码作为验证信息



```
git clone http://{邮箱/用户名}:{密码}@xxx.git
```

注意使用邮箱做验证信息时，邮箱中的@要进行转码为%40，后面分隔url和验证信息的@无需转码, 使用用户名则没有关系

例：

```
// 通过邮箱clone项目
git clone http://321321321%40qq.com:123123123@xxx.git
// 通过用户名clone项目
git clone http://myname:123123123@xxx.git
```



```
桌面 — zsh — 80x24
[zhaoxingrui@ZhaoXingRuis-MacBook-Pro Desktop % git clone http://zhaoxr429%40163.com:
com: [REDACTED]@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git
Cloning into 'todo-list-backend'...
warning: redirecting to https://seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-backend.git/
warning: You appear to have cloned an empty repository.
[zhaoxingrui@ZhaoXingRuis-MacBook-Pro Desktop % git clone http://zhaoxr429: [REDACTED]@seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git
Cloning into 'todo-list-frontend'...
warning: redirecting to https://seecoder-gitlab-gitlab.seec.seecoder.cn/zhaoxr429/todo-list-frontend.git/
warning: You appear to have cloned an empty repository.
zhaoxingrui@ZhaoXingRuis-MacBook-Pro Desktop %
```

分别将示例前后端代码复制到前后端文件夹内，**注意不要把老的.git文件复制过来，只复制项目文件**。将项目上传至各自的仓库，在新的仓库上协作开发。

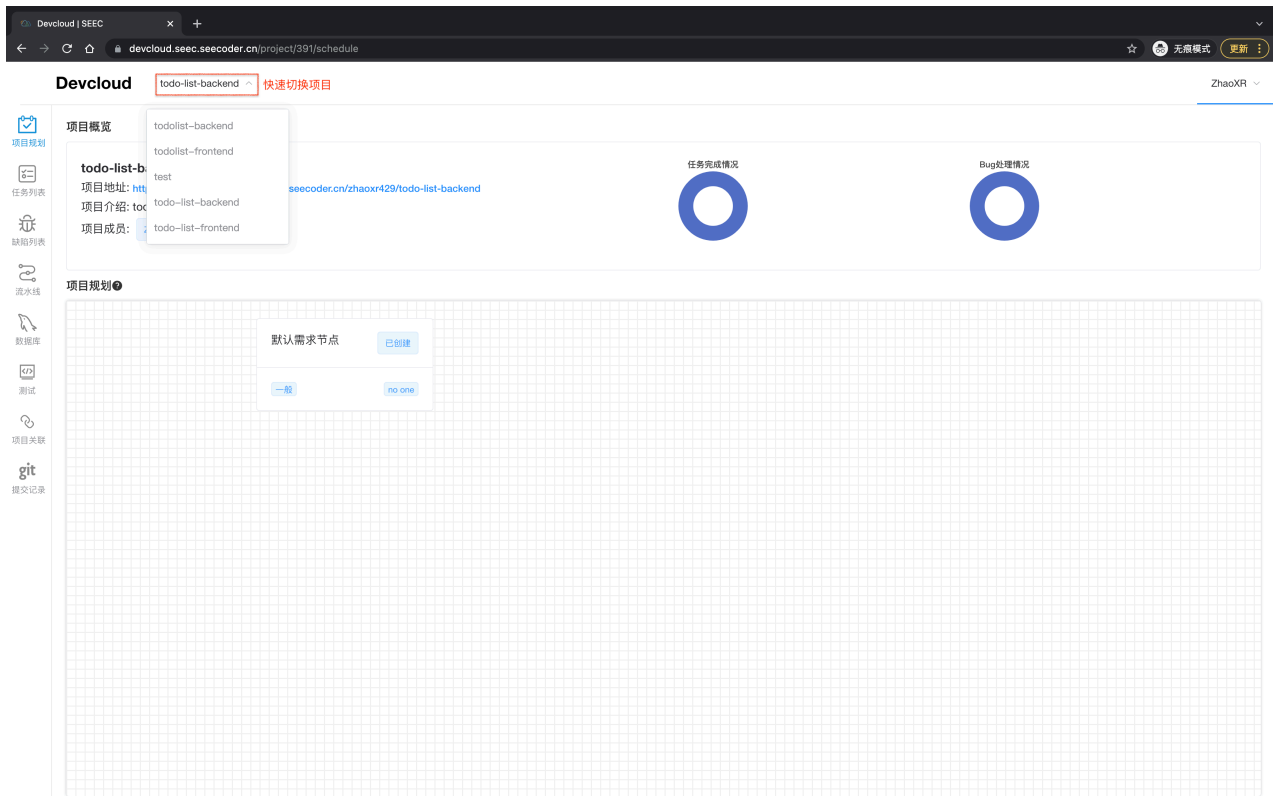
```
git add .
git commit -m "init: 项目初始化"
git push
```

```
todo-list-backend -- zsh -- 80x42
[zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-backend % git add .
[zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-backend % git commit -m "init: 后端项目初始化"
[master (root-commit) 2eb5f22] init: 后端项目初始化
23 files changed, 709 insertions(+)
create mode 100644 .DS_Store
create mode 100644 .gitignore
create mode 100644 .mvn/wrapper/maven-wrapper.jar
create mode 100644 .mvn/wrapper/maven-wrapper.properties
create mode 100644 mvnw.cmd
create mode 100644 pom.xml
create mode 100644 src/.DS_Store
create mode 100644 src/main/.DS_Store
create mode 100644 src/main/java/com/example/demo/DemoApplication.java
create mode 100644 src/main/java/com/example/demo/controller/TaskController.java
a
create mode 100644 src/main/java/com/example/demo/dao/TaskMapper.java
create mode 100644 src/main/java/com/example/demo/model/po/TaskPO.java
create mode 100644 src/main/java/com/example/demo/model/vo/TaskVO.java
create mode 100644 src/main/java/com/example/demo/service/TaskService.java
create mode 100644 src/main/java/com/example/demo/service/impl/TaskServiceImpl.java
java
create mode 100644 src/main/resources/.DS_Store
create mode 100644 src/main/resources/application.properties
create mode 100644 src/main/resources/application.yml
create mode 100644 src/main/resources/mapper/TaskMapper.xml
create mode 100644 src/main/resources/sql/task.sql
create mode 100644 src/test/java/com/example/demo/DemoApplicationTests.java
create mode 100644 src/test/java/com/example/demo/mapper/TaskMapperTest.java
create mode 100644 src/test/java/com/example/demo/util/RandomUtil.java
[zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-backend % git push
warning: redirecting to https://seecoder-gitlab-gitlab.seecoder.cn/zhaoxr429/todo-list-backend.git/
Enumerating objects: 50, done.
Counting objects: 100% (50/50), done.
Delta compression using up to 12 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (50/50), 62.08 KiB | 8.87 MiB/s, done.
Total 50 (delta 3), reused 0 (delta 0), pack-reused 0
To http://seecoder-gitlab-gitlab.seecoder.cn/zhaoxr429/todo-list-backend.git
* [new branch]      master -> master
```

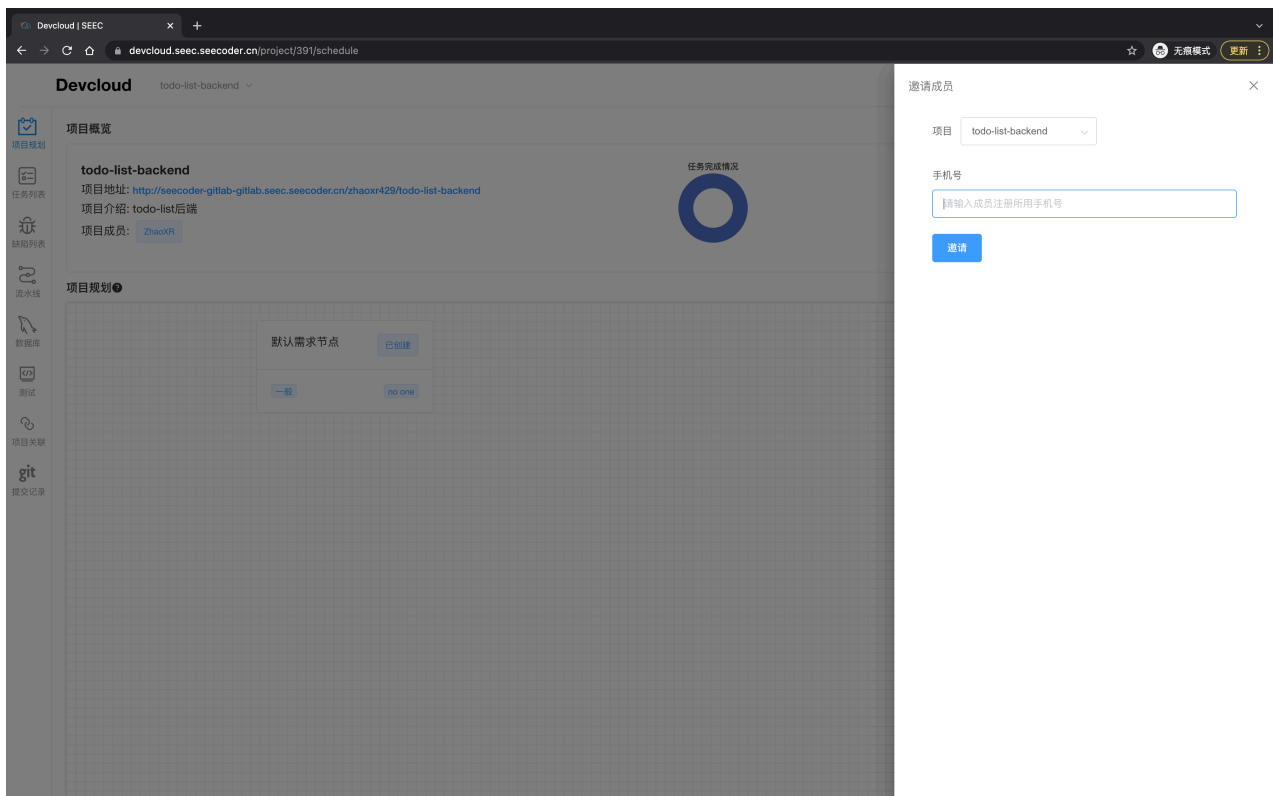
```
todo-list-frontend — zsh — 80x39
zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-frontend % git add .
zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-frontend % git commit -m "init: 前端项目初始化"
[master (root-commit) 74bb26b] init: 前端项目初始化
21 files changed, 9718 insertions(+)
create mode 100644 .browserslistrc
create mode 100644 .eslintrc.js
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 babel.config.js
create mode 100644 package.json
create mode 100644 public/favicon.ico
create mode 100644 public/index.html
create mode 100644 src/App.vue
create mode 100644 src/assets/logo.png
create mode 100644 src/components/Header.vue
create mode 100644 src/components/HelloWorld.vue
create mode 100644 src/components/Item.vue
create mode 100644 src/components/List.vue
create mode 100644 src/main.js
create mode 100644 src/router/index.js
create mode 100644 src/store/index.js
create mode 100644 src/views/About.vue
create mode 100644 src/views/Home.vue
create mode 100644 vue.config.js
create mode 100644 yarn.lock
zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-frontend % git push
warning: redirecting to https://seecoder-gitlab-gitlab.seecoder.cn/zhaoxr429/todo-list-frontend.git/
Enumerating objects: 30, done.
Counting objects: 100% (30/30), done.
Delta compression using up to 12 threads
Compressing objects: 100% (26/26), done.
Writing objects: 100% (30/30), 178.16 KiB | 8.91 MiB/s, done.
Total 30 (delta 0), reused 0 (delta 0), pack-reused 0
To http://seecoder-gitlab-gitlab.seecoder.cn/zhaoxr429/todo-list-frontend.git
 * [new branch]      master -> master
zhaoxingrui@ZhaoXingRuis-MacBook-Pro todo-list-frontend %
```

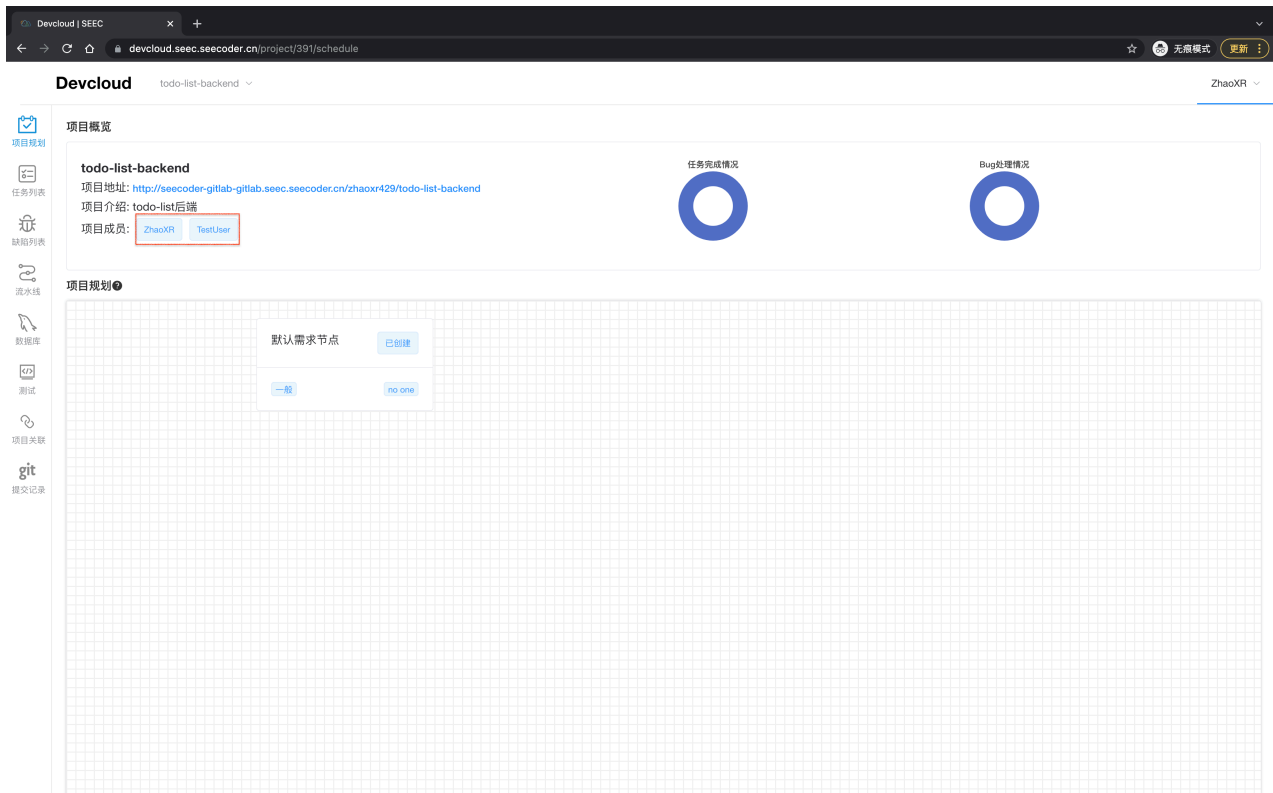
1.3 邀请项目成员

点击首页的项目卡片进入具体的项目中，点击左上角项目名称右边的下拉框，可以快速选择其他项目



点击右上角id右边的下拉框，点击邀请成员，填写手机号邀请组员





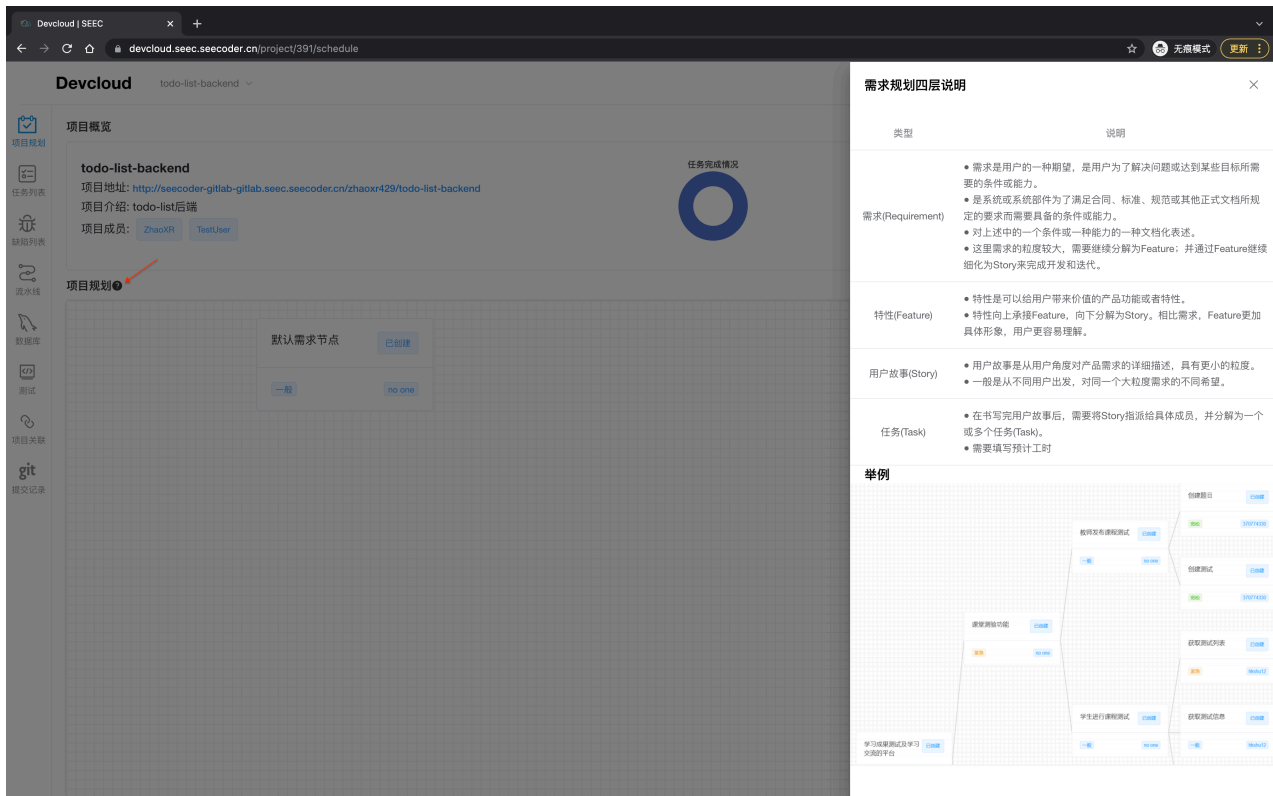
2. 项目规划-需求

2.1 功能介绍

需求为一个深度为4的树，用户可以自由新增删除需求节点

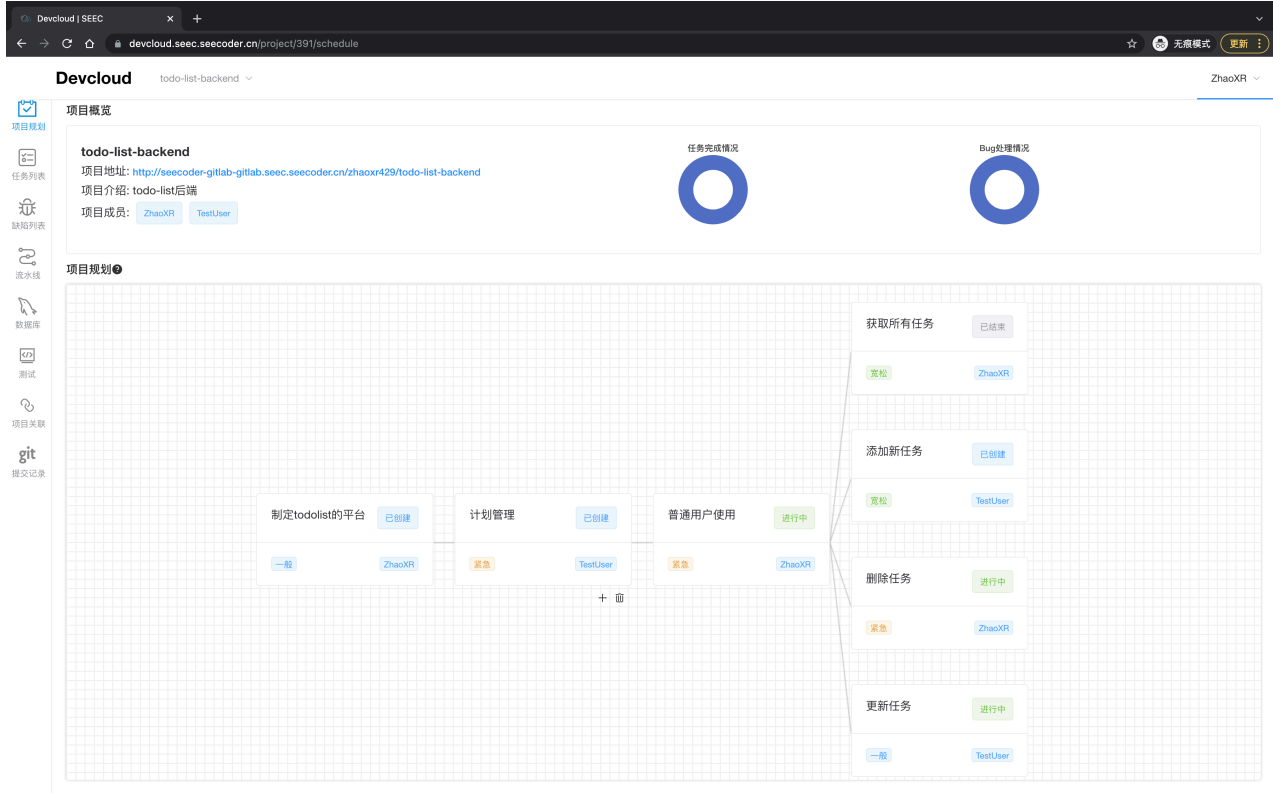
每一层对应一类需求，从左到右/从粗粒度到细粒度，对应着 Requirement、Feature、Story、Task 四类需求，每个需求节点可以修改名称、设置紧急度、工时、指派负责人等。

点击项目规划字段右边的 问号符号 即可查看需求树的4层说明及举例



2.2 创建需求规划树

将鼠标置于卡片后在卡片下方显示添加按钮，点击后自动创建一个默认节点，下图是todo-list后端项目的需求规划树，请大家在项目内自行完成需求规划树，并设置紧急度，工时，负责人等



task是最小的可分配任务粒度

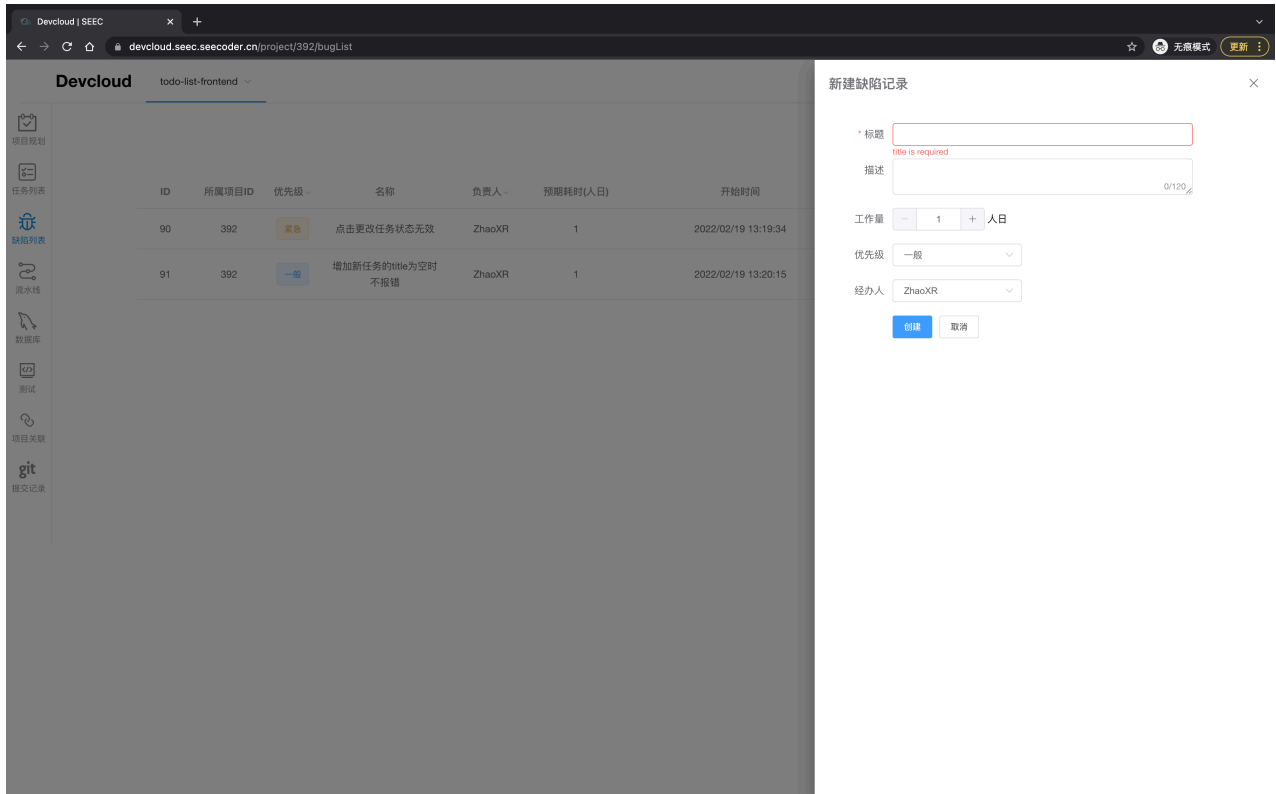
四类节点都可以更新内容，点击卡片的编辑按钮即可进行更新

3. bug记录功能

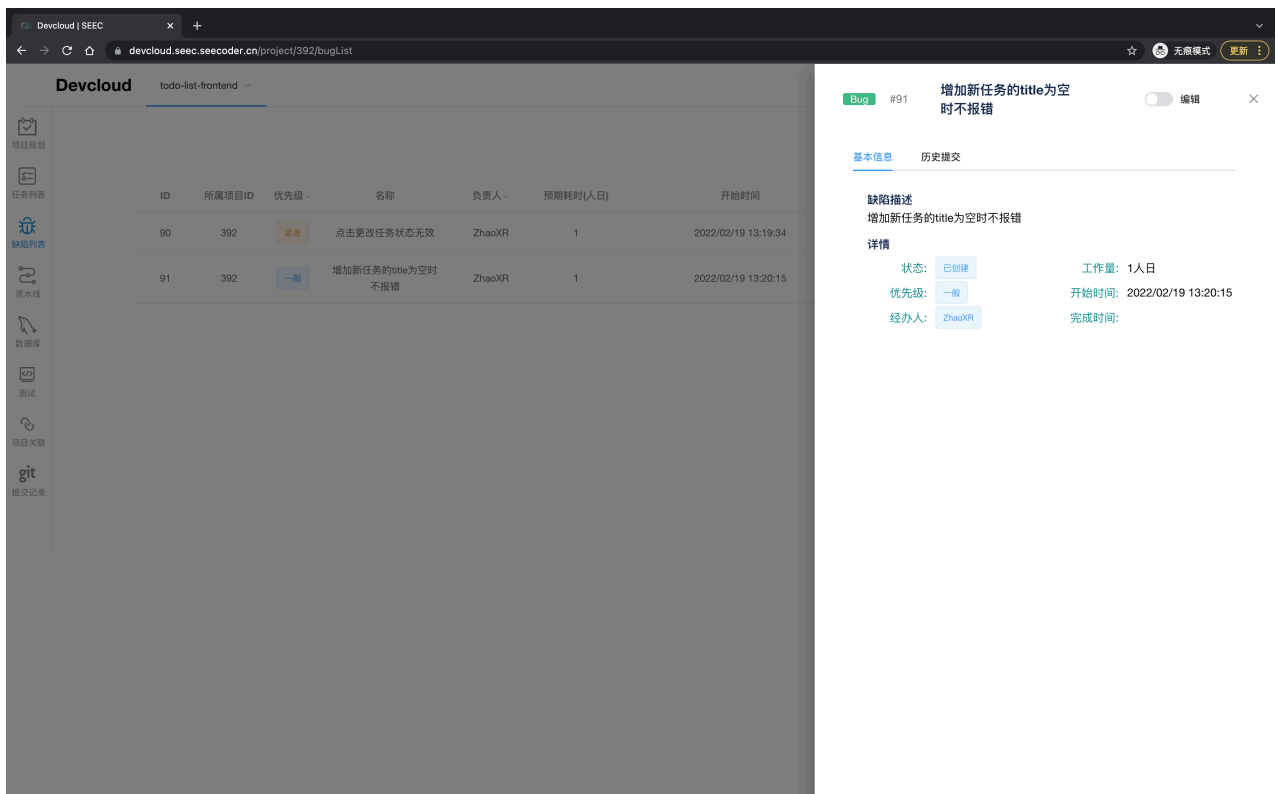
3.1 记录项目缺陷

导航栏第三项 **缺陷列表** 可以记录项目中出现的bug

点击新建，可以将使用的时候遇到的Bug记录下来，设置负责人和处理时间



点击具体的一项可以查看缺陷的详情并可以修改配置



在给出的前端代码中存在两个缺陷，如上图所示，**请大家在前端项目的缺陷列表中建立相应的bug并将代码中的bug解决**

4. 流水线及部署功能

4.1 功能介绍

导航栏第四项 **流水线**

集群资源有限，请不要部署无关的应用！

部署功能分为两个部分

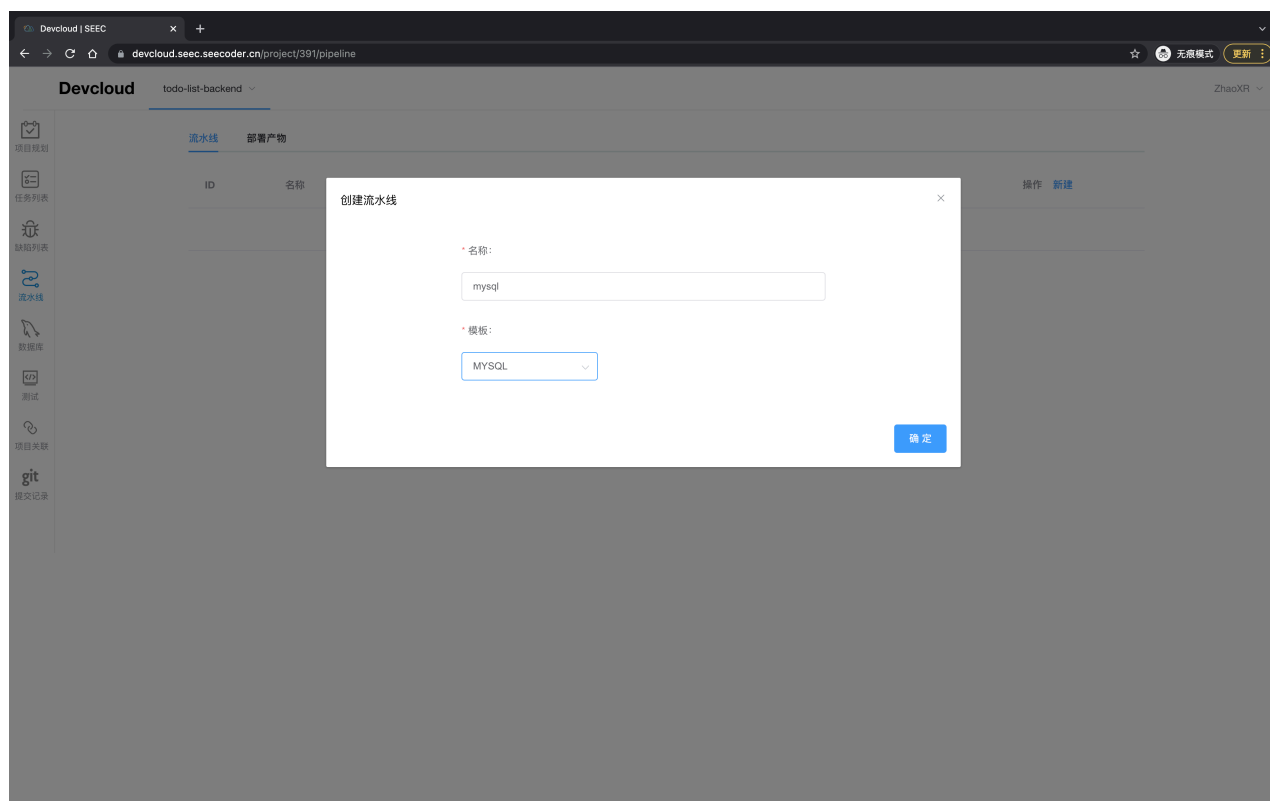
- 流水线：负责部署的流程配置
- 部署产物：负责部署实例的管理

注：mysql和后端实例在后端项目中创建，前端实例在前端项目中创建

4.2 创建后端项目使用的mysql

4.2.1 创建mysql

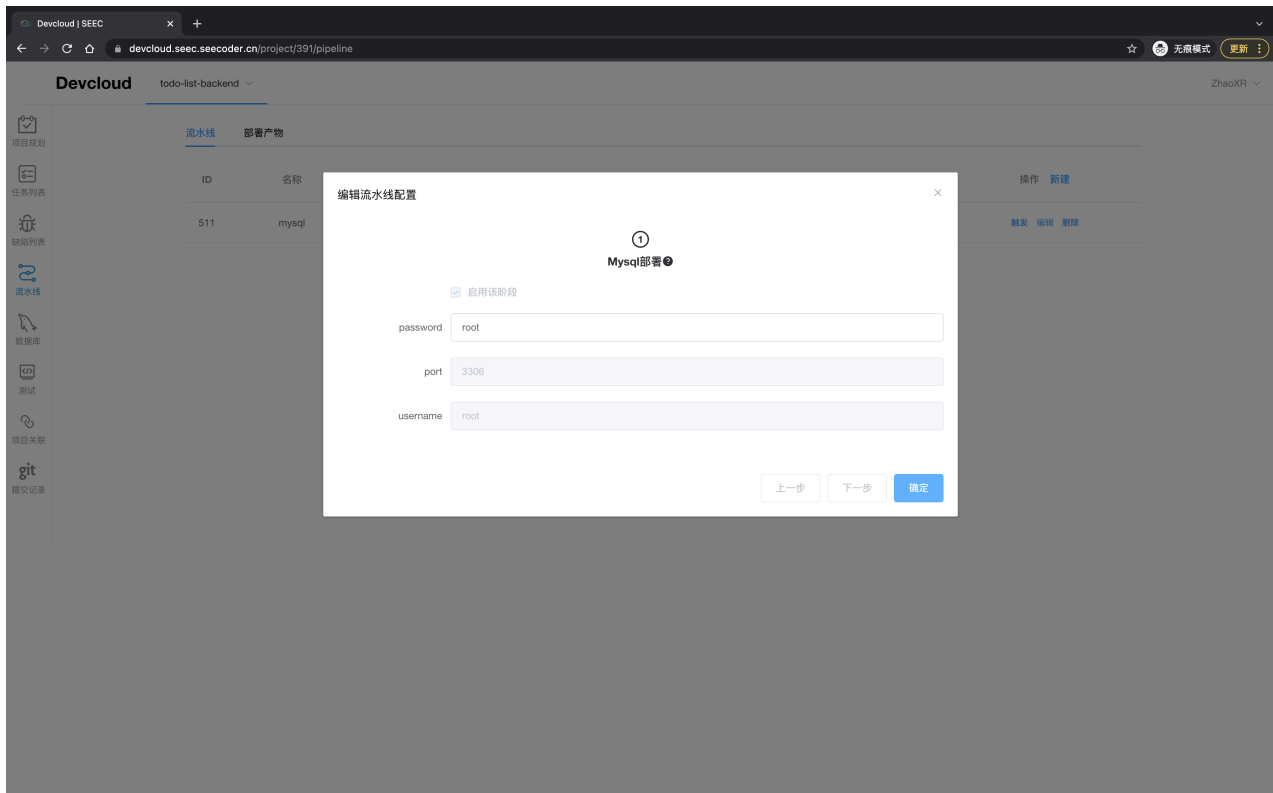
点击新建，输入流水线名称，这里选择的模板是 **MYSQL**



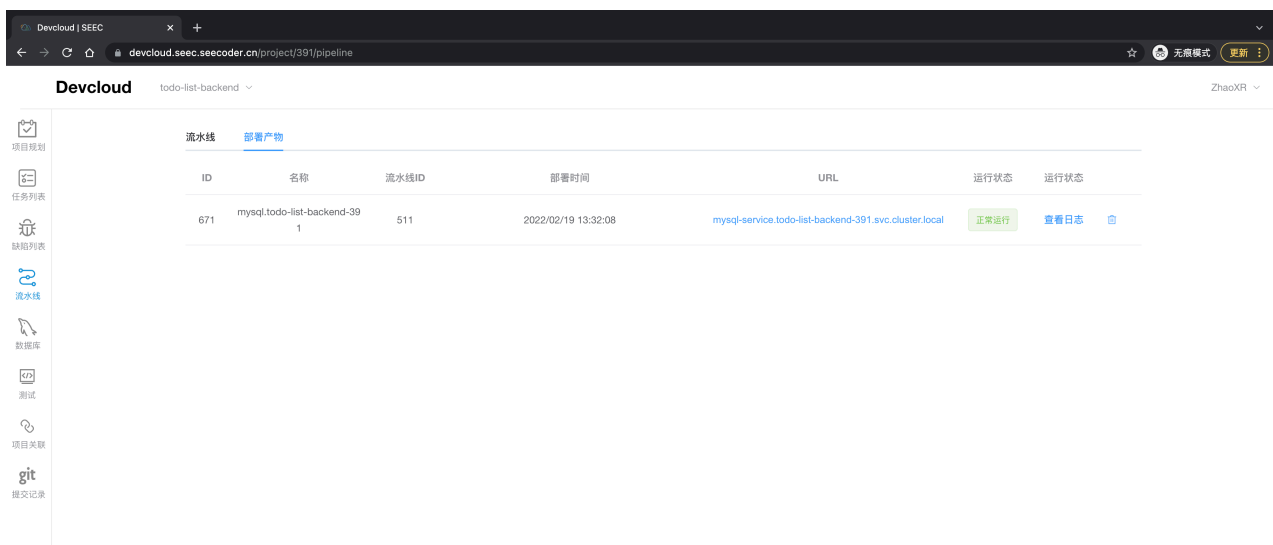
注意流水线名称不能重复, 流水线名称只能为以小写英文字母开头，且由英文字母和‘-’组成的字符串，长度为3-30

这里的不能重复是指同一个项目内的流水线名称不能重复

创建完成后点击编辑，配置mysql的管理员密码，端口默认使用3306



编辑完成之后，点击确定，再点击触发进行部署，耐心等待一段时间部署结束，在部署产物中可以看到mysql部署产物正常运行



4.2.2 mysql数据库、表、数据初始化

点击导航栏第五项 **数据库**

注意，这是个新加的功能，出于安全性考虑，集群无法直接暴露mysql的端口，只能用http请求的方式执行

sql，线上使用体验可能有点差。所以尽可能地在本地调试好应用，防止线上查bug的时候比较麻烦

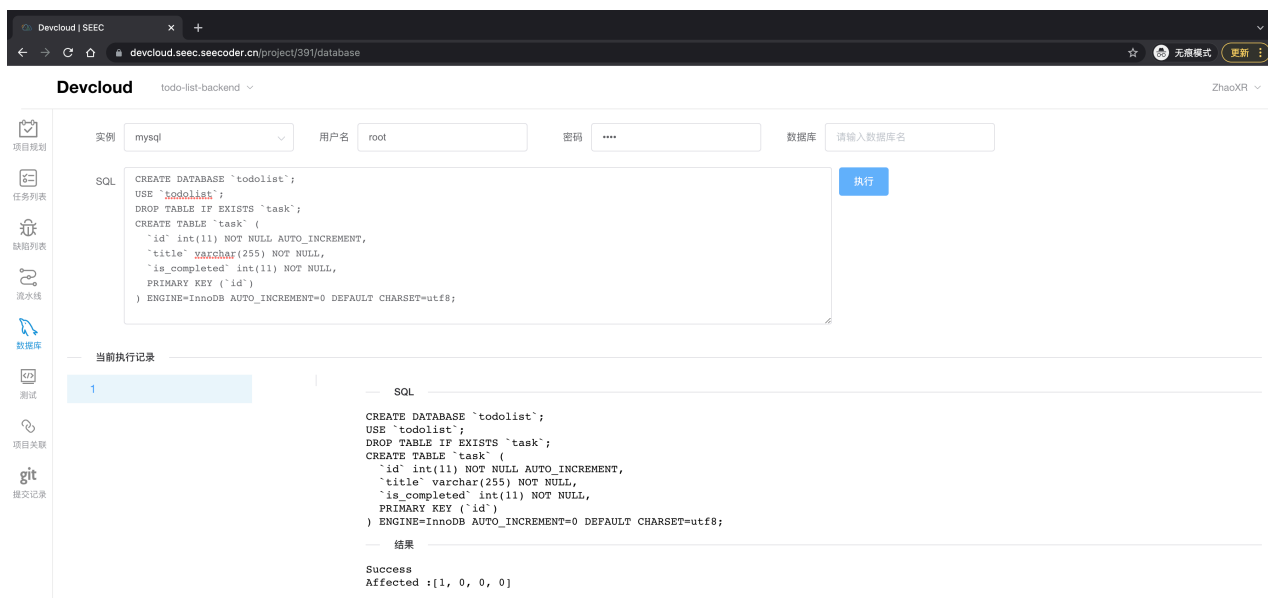
下拉选择刚刚部署的Mysql实例，填入用户名密码（密码是刚刚新建的mysql流水线中配置的密码）
数据库名称空着(因为你还没建)

将建表sql语句复制到下面执行

```
CREATE DATABASE `todolist`;
USE `todolist`;
DROP TABLE IF EXISTS `task`;
CREATE TABLE `task` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `is_completed` int(11) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=0 DEFAULT CHARSET=utf8;
```

执行结果如下，数字代表每一句sql的影响（新增/删除）行数

有些为0，是因为有些sql本来没有新增或删除



The screenshot shows the Devcloud interface for a project named 'todo-list-backend'. The top bar includes the Devcloud logo and the project name. The sidebar on the left contains various navigation icons. The main area is divided into two sections: a SQL editor and an execution results panel. The SQL editor contains the following commands:

```
CREATE DATABASE `todolist`;
USE `todolist`;
DROP TABLE IF EXISTS `task`;
CREATE TABLE `task` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `is_completed` int(11) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=0 DEFAULT CHARSET=utf8;
```

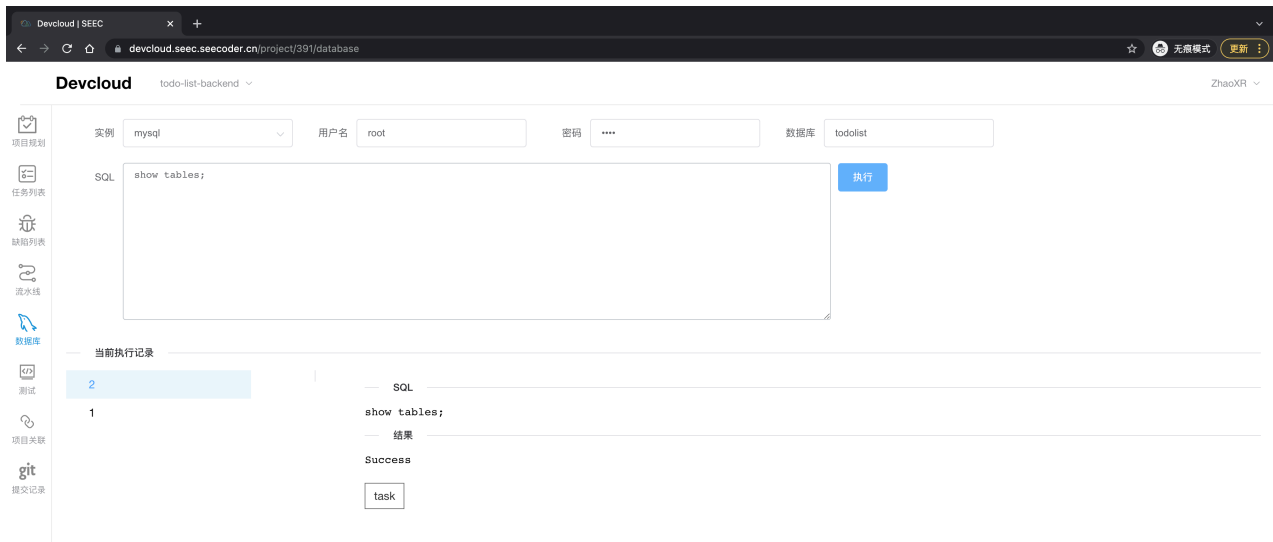
The execution results panel shows a table with 1 row, 0 columns, and 0 affected rows. The table is titled '当前执行记录' (Current Execution Record). The results are as follows:

SQL	结果
CREATE DATABASE `todolist`; USE `todolist`; DROP TABLE IF EXISTS `task`; CREATE TABLE `task` (`id` int(11) NOT NULL AUTO_INCREMENT, `title` varchar(255) NOT NULL, `is_completed` int(11) NOT NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB AUTO_INCREMENT=0 DEFAULT CHARSET=utf8;	Success Affected :[1, 0, 0, 0]

初始化完成后，你就可以填入你的数据库名称来查询了，这里填入刚刚创建的数据库名 `todolist`

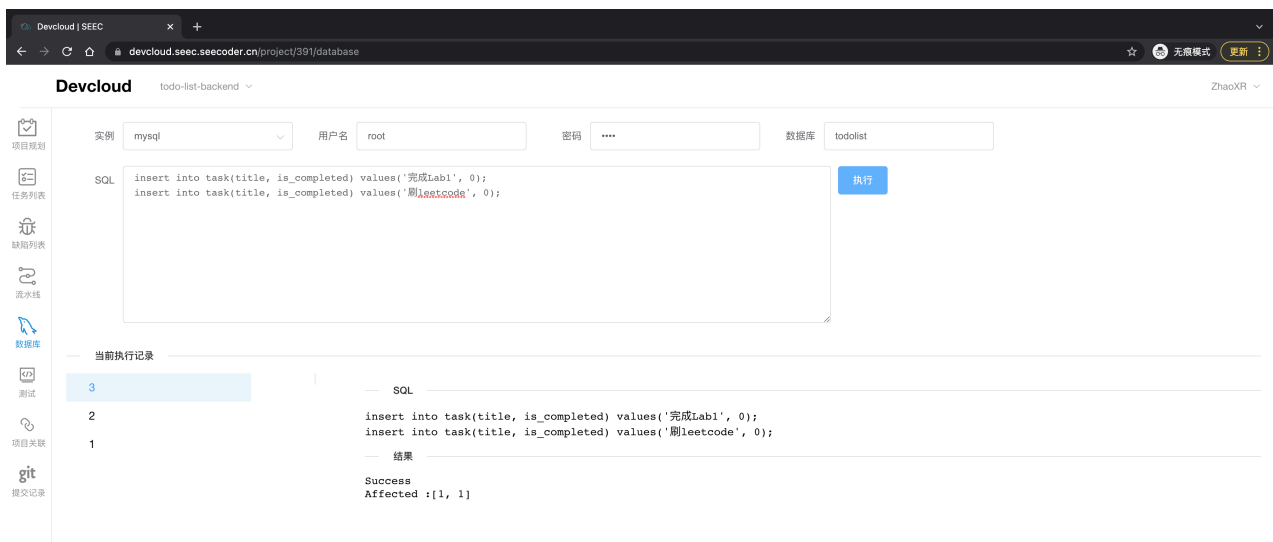
示例：填入我们刚创建的数据库todolist, 执行一下show tables，能看见我们刚刚创建的表 task

```
show tables;
```



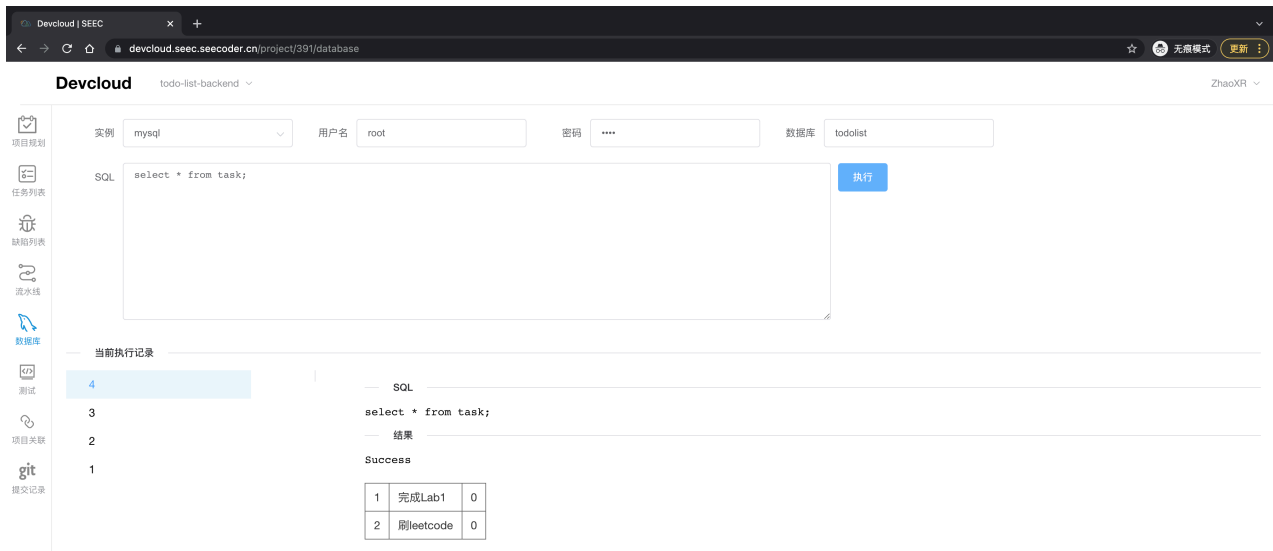
可以再插入两条数据，插入的sql语句及结果如下：

```
insert into task(title, is_completed) values('完成Lab1', 0);  
insert into task(title, is_completed) values('刷leetcode', 0);
```



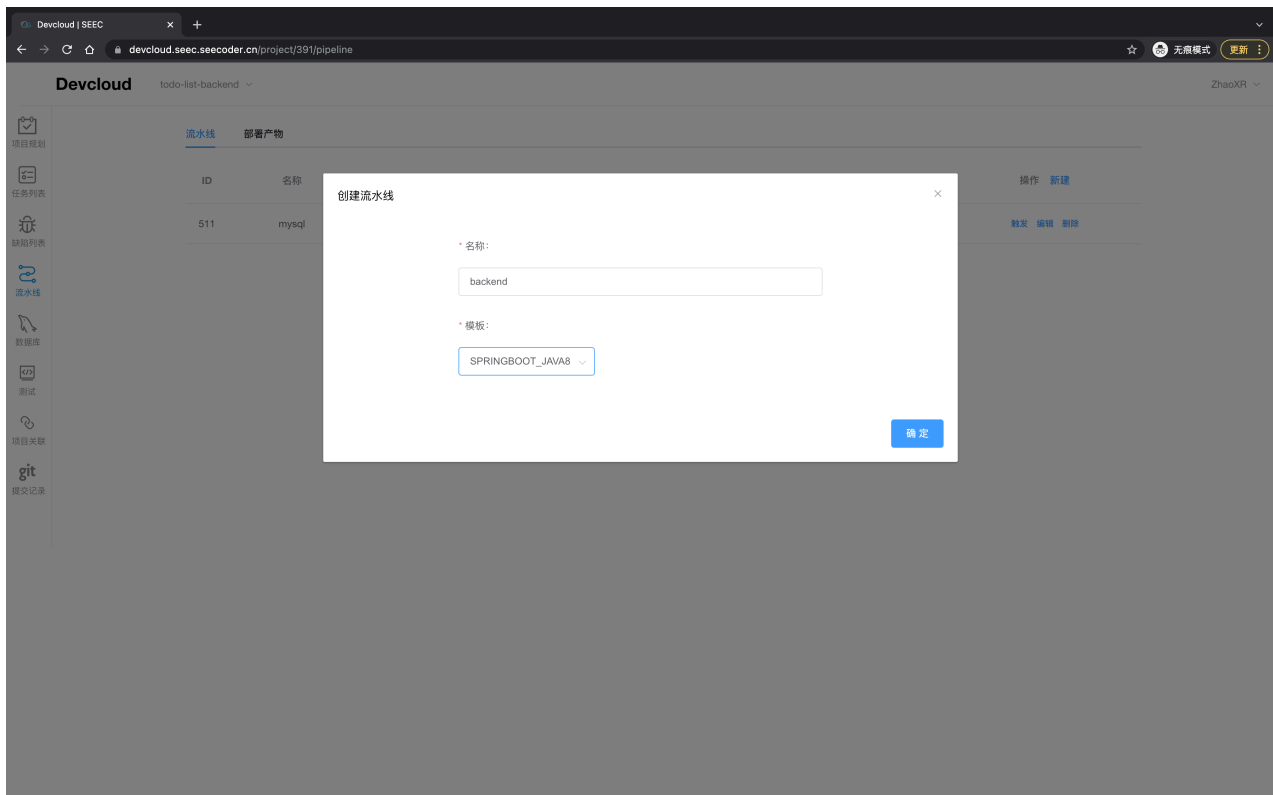
这时才查询一下刚刚插入的数据，查询语句及结果如下：

```
select * from task;
```



4.3 创建SpringBoot后端实例

回到流水线界面，点击新建，输入流水线名称，选择SPRINGBOOT_JAVA8模板（注意上述提到的流水线名称规范）

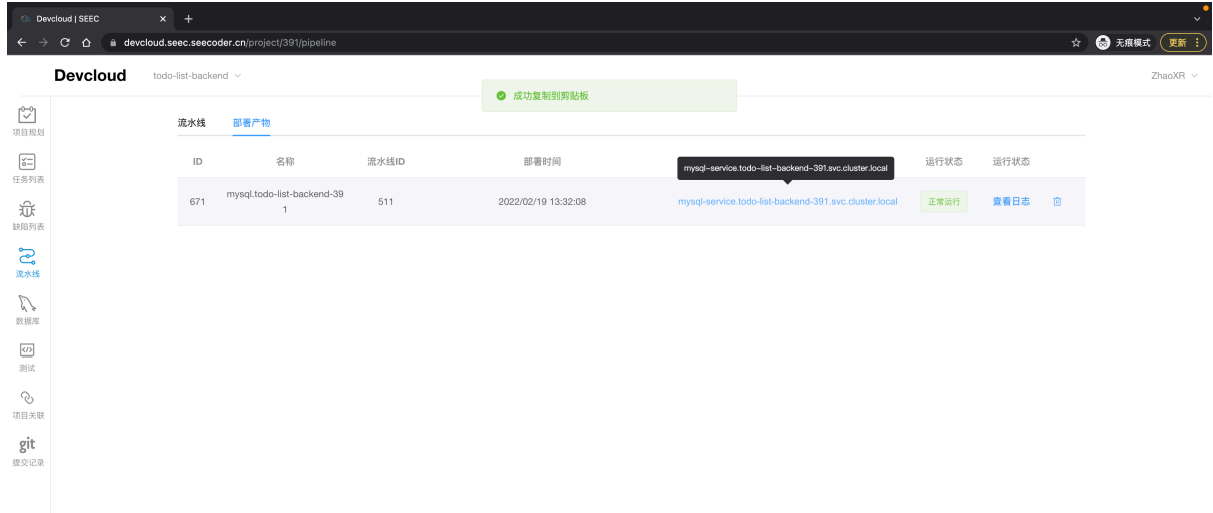


4.3.1 代码配置

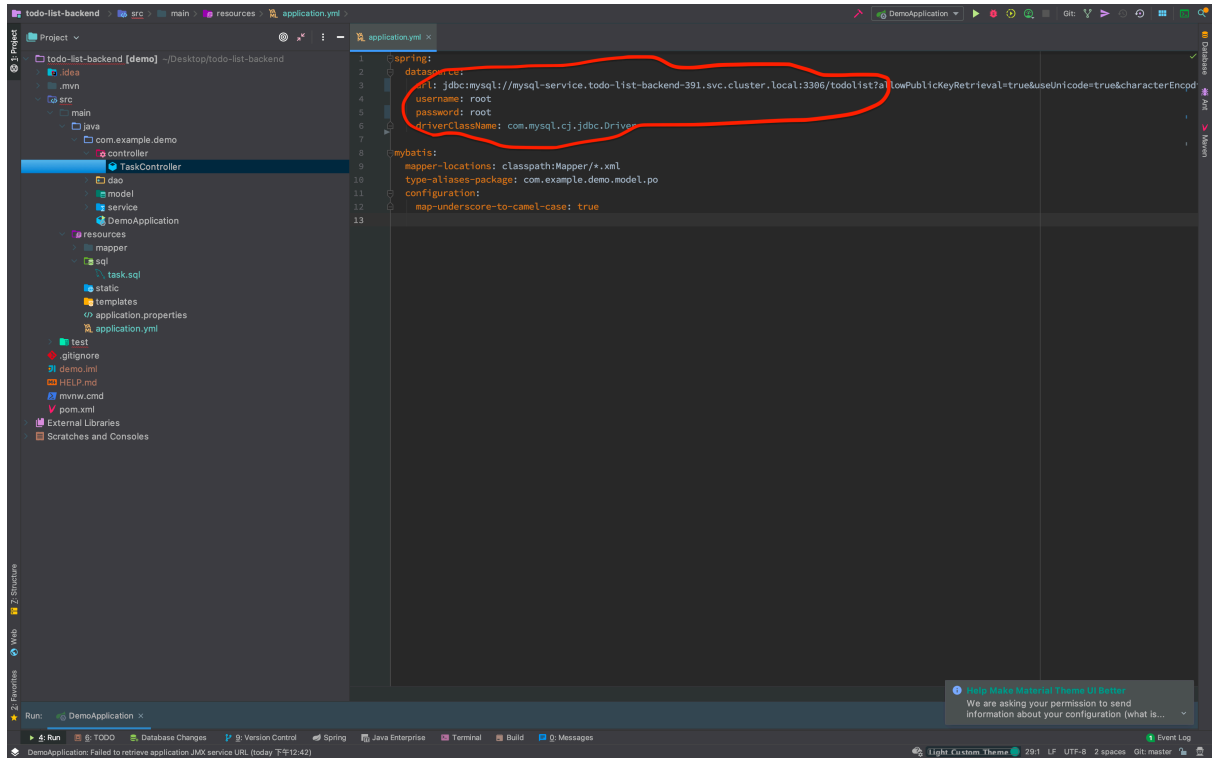
注意：更改后端mysql配置信息后记得push变更！！

1. 将后端代码连接的mysql配置与线上保持一致x

修改后端配置文件 `application.yml`：在部署产物的mysql一栏点击URL即可复制到剪切板，注意数据库名要与刚刚新建的数据库名保持一致，用户名为root，密码是刚刚数据库设置的密码



```
spring:
  datasource:
    url: jdbc:mysql://mysql-service.todo-list-backend-391.svc.cluster.local:3306/todolist?allowPublicKeyRetrieval=true&useUnicode=true&characterEncoding=UTF-8&serverTimezone=GMT%2B8&useSSL=false
    username: root
    password: root
    driverClassName: com.mysql.cj.jdbc.Driver
```



2. 更改完之后记得push变更

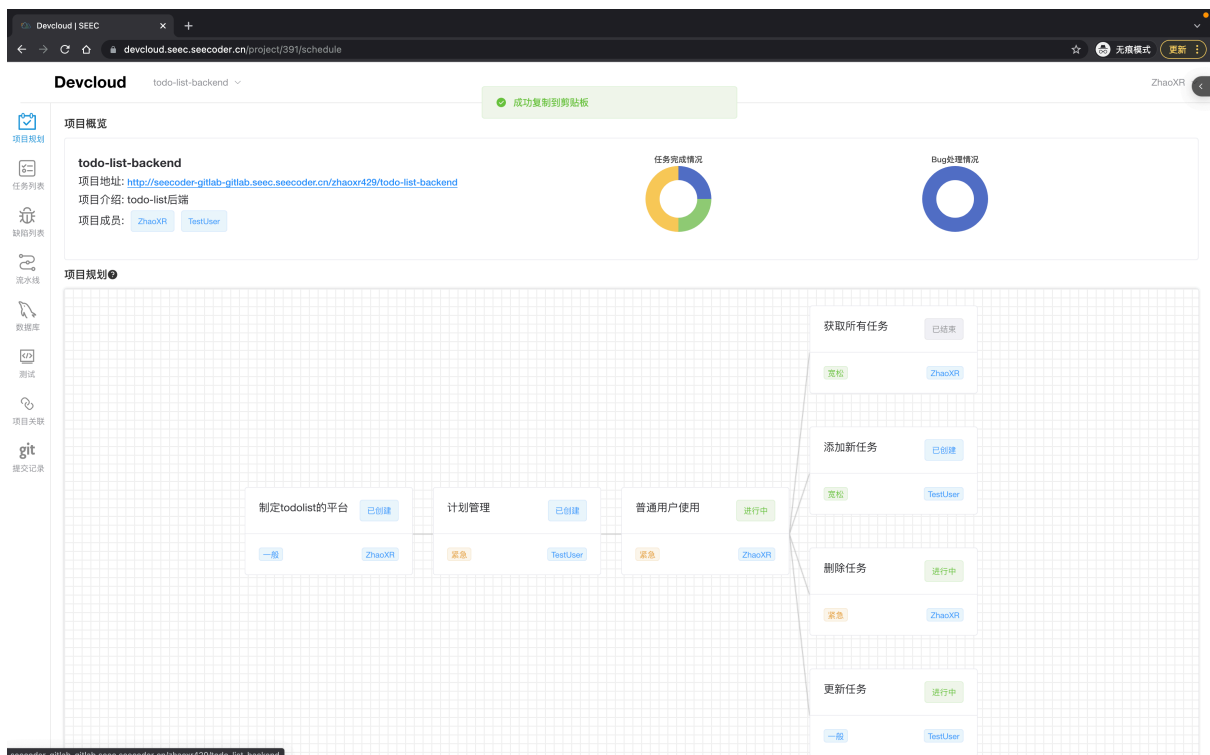
4.3.2 流水线配置

点击后端流水线编辑

4.3.2.1 Java应用构建

1. repoUrl: 项目地址url用于clone, 不需要上文的验证信息, 请在结尾加上.git

可以在项目规划中项目概览点击项目地址进行复制



2. branchName: 需要用于部署的分支, 请确保这条分支的应用可以正确运行, 默认是master分支

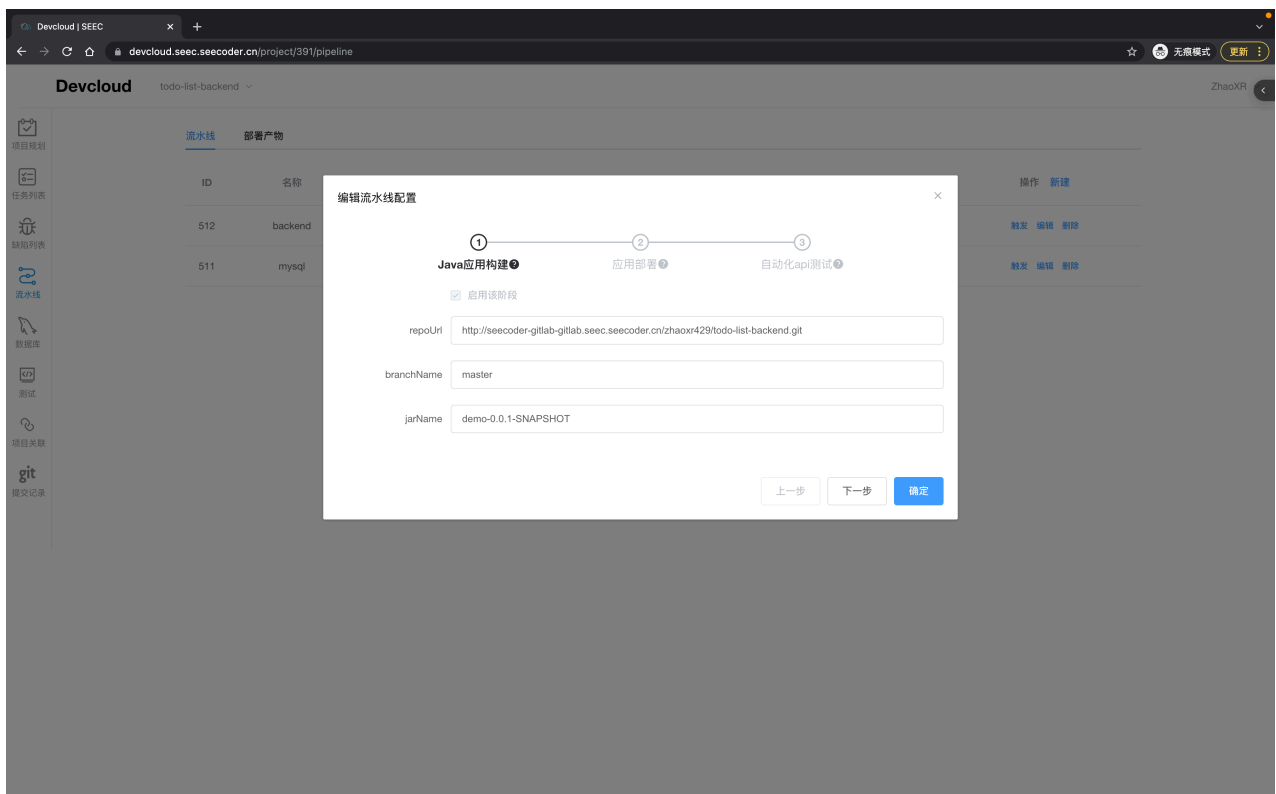
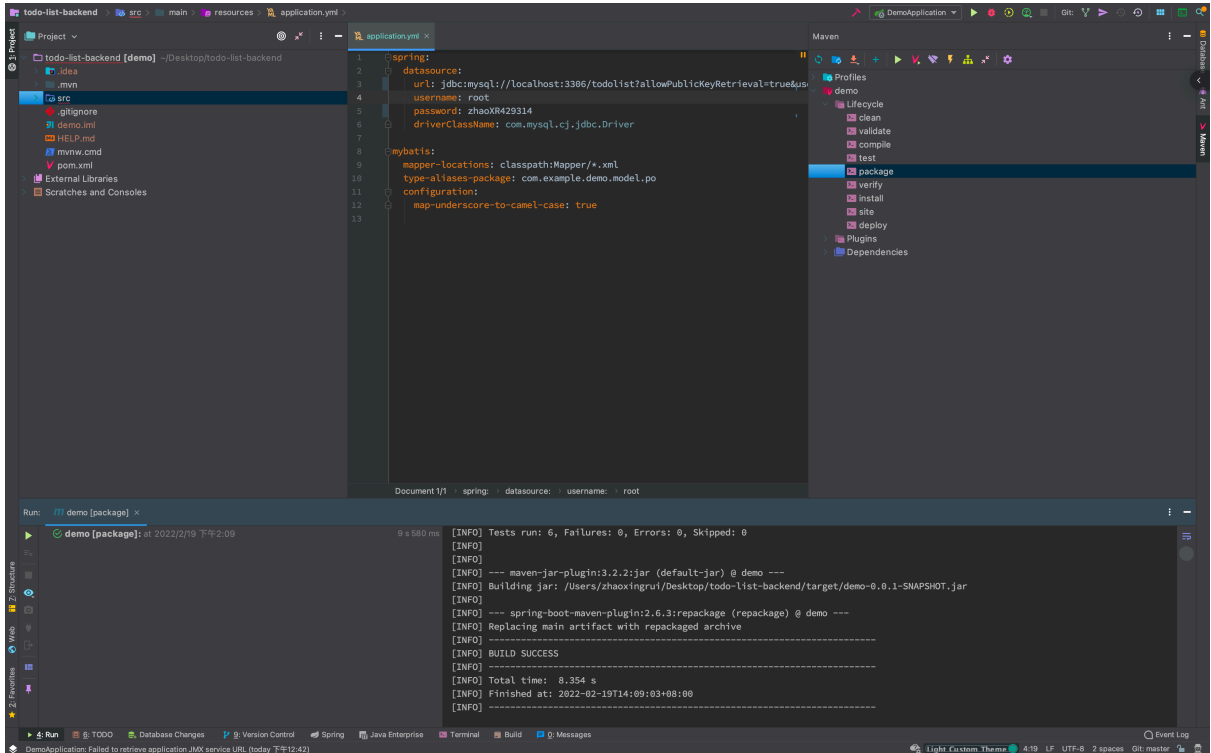
查看分支的方式：

1. IDEA右下角
2. 在项目根目录使用git branch命令查看当前分支

3. jarName：务必要和你打包出来的jar包名字一致，不需要.jar后缀

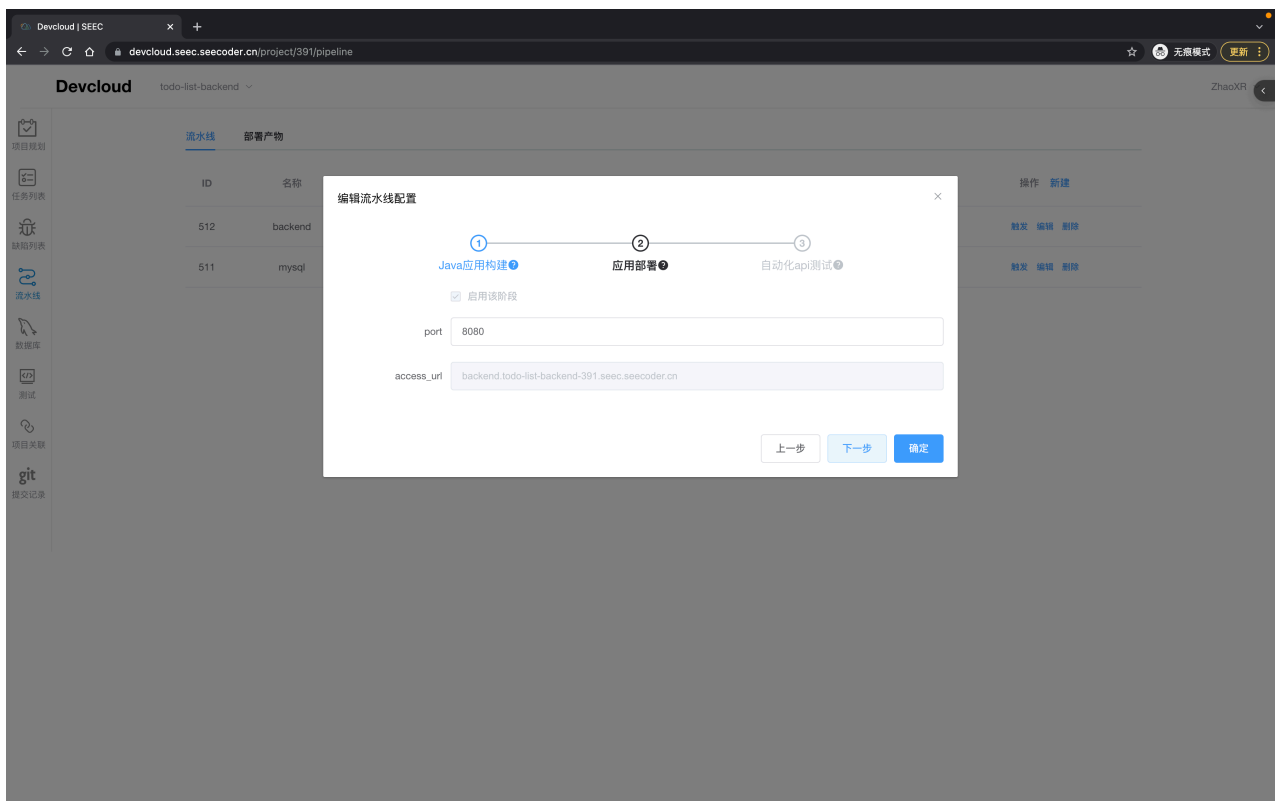
如：打包出来的应用叫demo-0.0.1-SNAPSHOT.jar，那么填入的值为demo-0.0.1-SNAPSHOT

不知道jar包名称的使用idea的maven package（或者在项目根目录使用mvn package命令）打包一次，到项目根目录/target下查看jar包名称



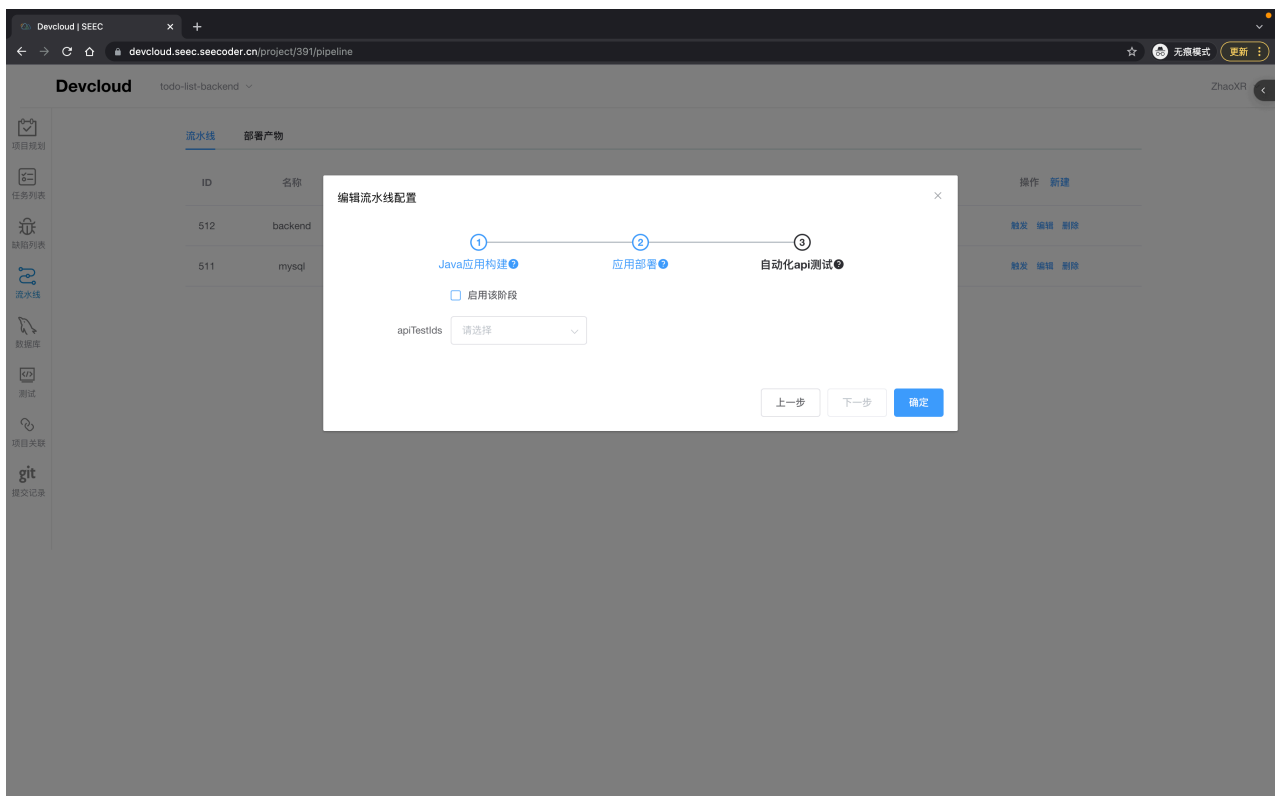
4.3.2.2 应用部署

port: 务必和你后端springboot application.yml配置暴露的端口一致（默认是8080）



4.3.2.3 自动化api测试

将启用该阶段勾除，暂不使用该阶段



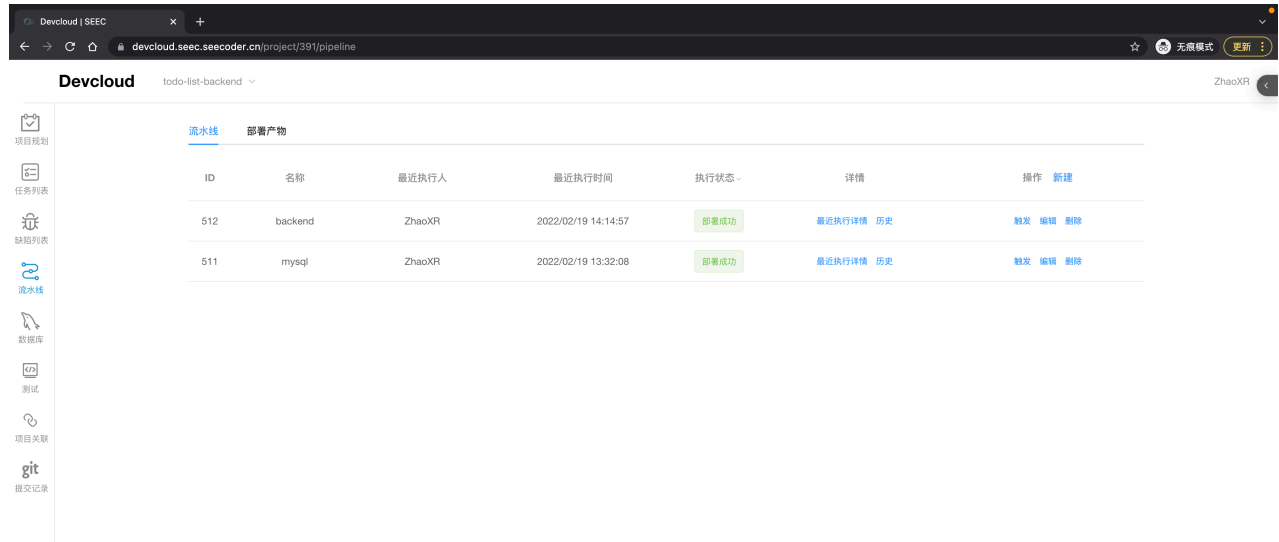
4.3.3 部署后端实例

点击触发进行部署

部署过程大概持续5-10min，请耐心等待

部署成功之前，部署产物会显示不存在（因为读取实例状态未读取到），请不要惊慌

部署成功如下所示：

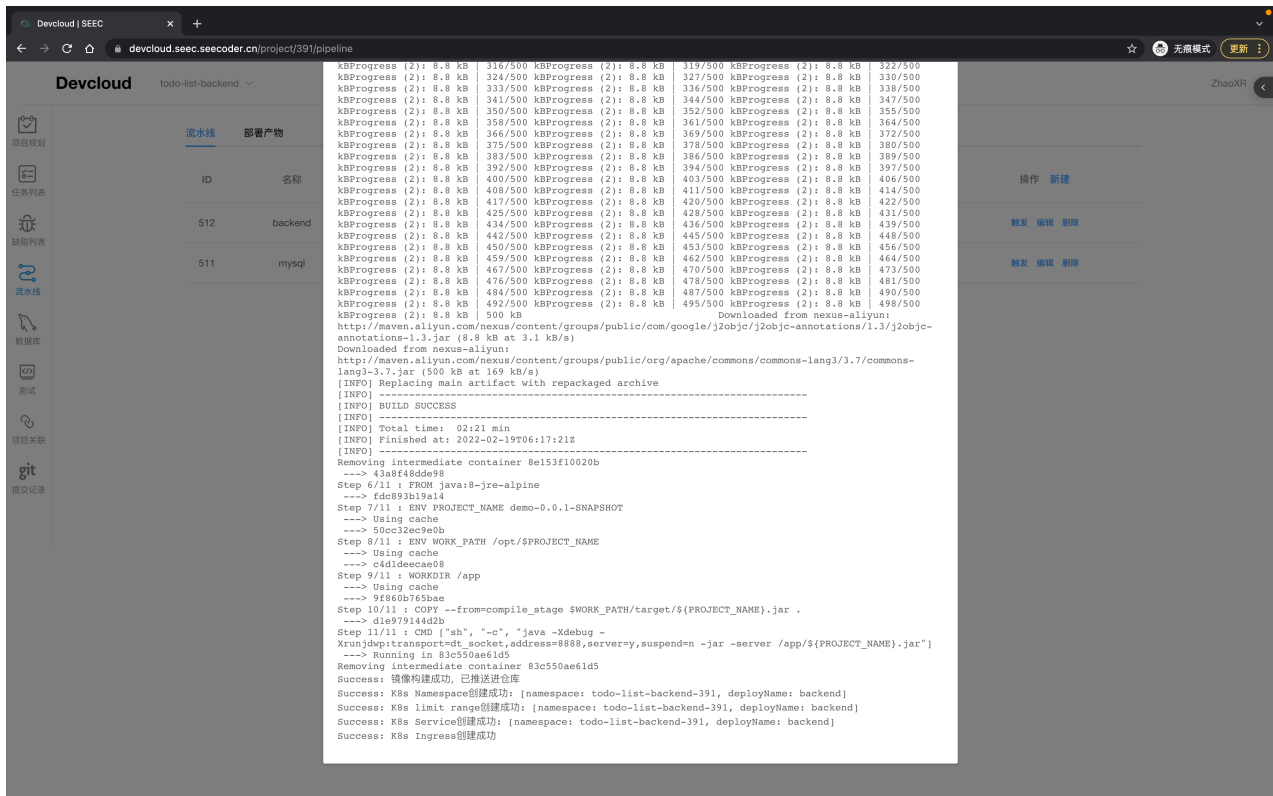


The screenshot shows the Devcloud web interface. The browser address bar indicates the URL is `devcloud.seecoder.cn/project/391/pipeline`. The page title is "Devcloud" and the breadcrumb is "todo-list-backend". The left sidebar contains navigation icons for "项目规划", "任务列表", "缺陷列表", "流水线", "数据库", "测试", "项目关联", and "git 提交记录". The main content area has two tabs: "流水线" (selected) and "部署产物". Below the tabs is a table with the following data:

ID	名称	最近执行人	最近执行时间	执行状态	详情	操作
512	backend	ZhaoXR	2022/02/19 14:14:57	部署成功	最近执行详情 历史	触发 编辑 删除
511	mysql	ZhaoXR	2022/02/19 13:32:08	部署成功	最近执行详情 历史	触发 编辑 删除

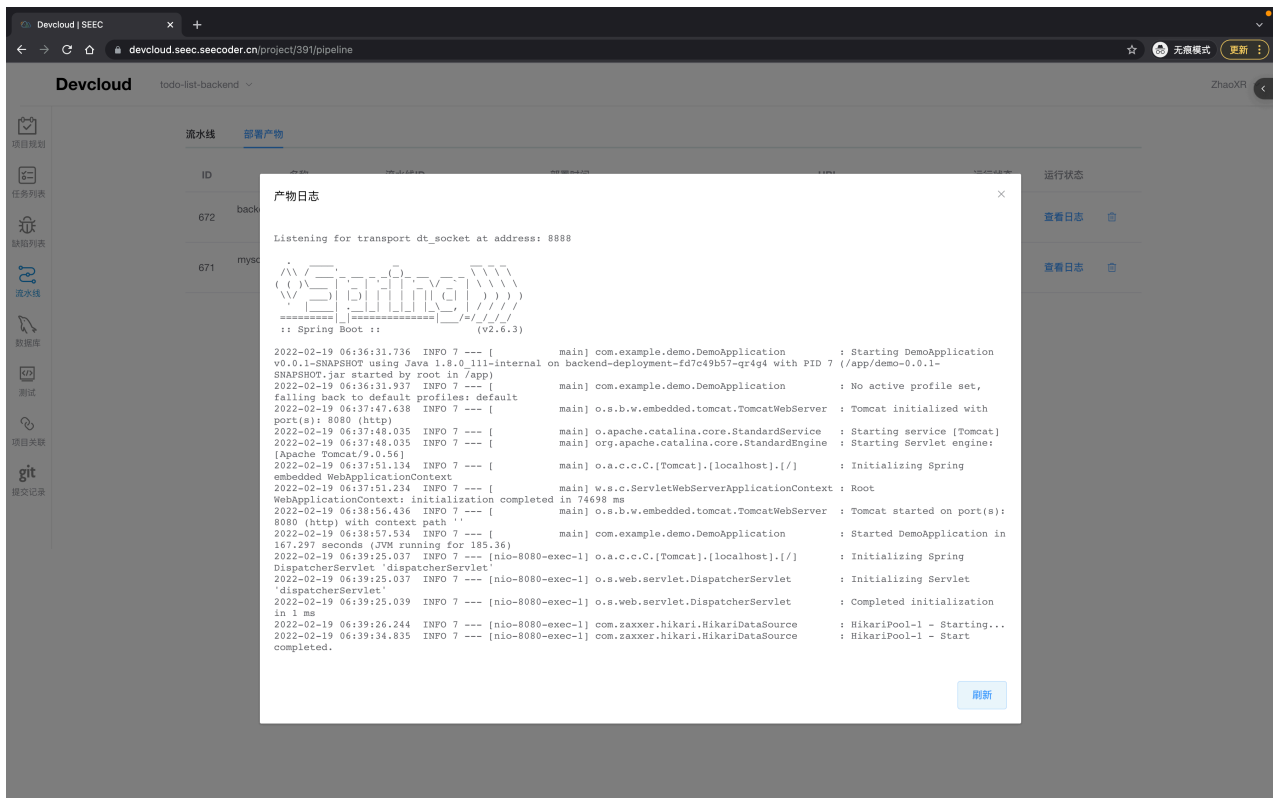
部署成功之后点击最后执行详情可以看见构建部署过程

部署失败请到也请到这里看部署日志，如果出现不能解决的问题联系助教



在部署实例产物界面，点击后端产物项中的查看日志可以查看实例日志

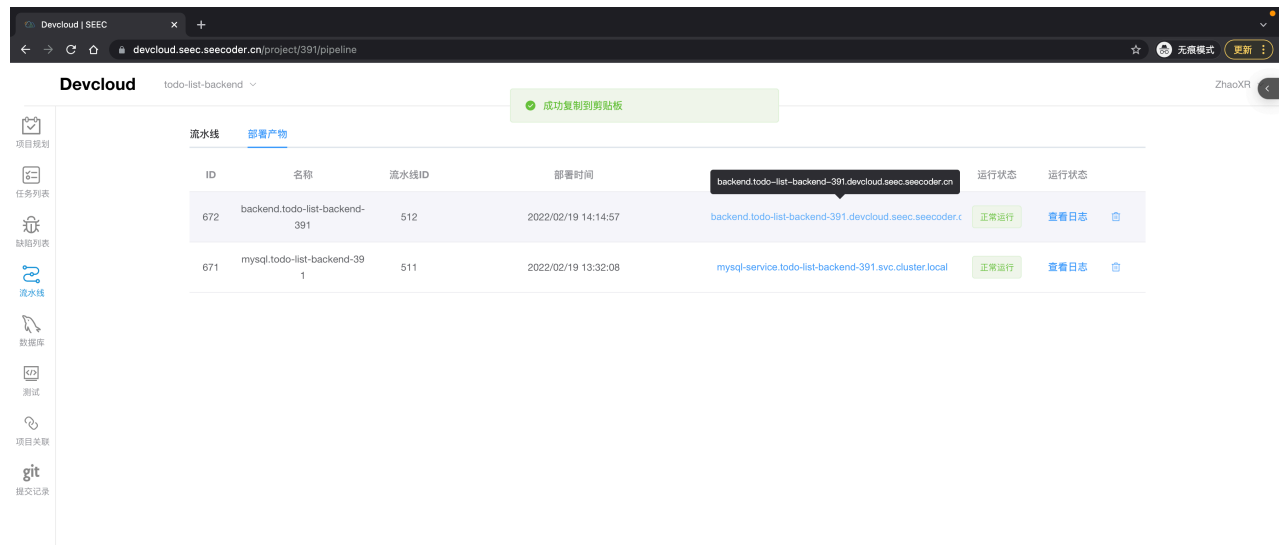
线上Debug主要在这个日志查询问题



4.3.5 测试

部署好之后，确认运行日志无异常之后，可以尝试下调用接口，确认后端是否正确运行。

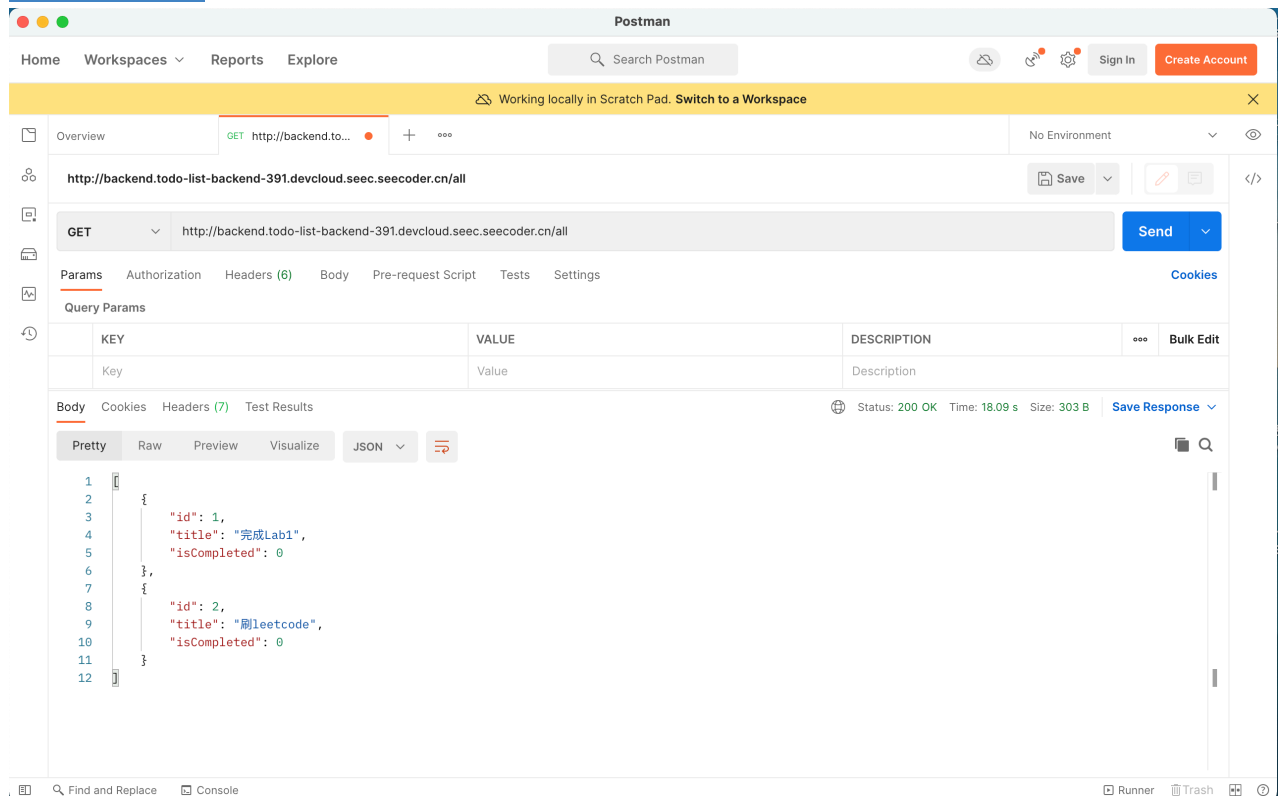
到部署产物界面点击后端部署产物的url即可进行复制



例如：backend.todo-list-backend-391.devcloud.seec.seecoder.cn

就通过这个地址使用http请求接口，注意请求url不用写端口号

使用curl、postman之类的工具进行接口测试：<http://backend.todo-list-backend-391.devcloud.seec.seecoder.cn/all>

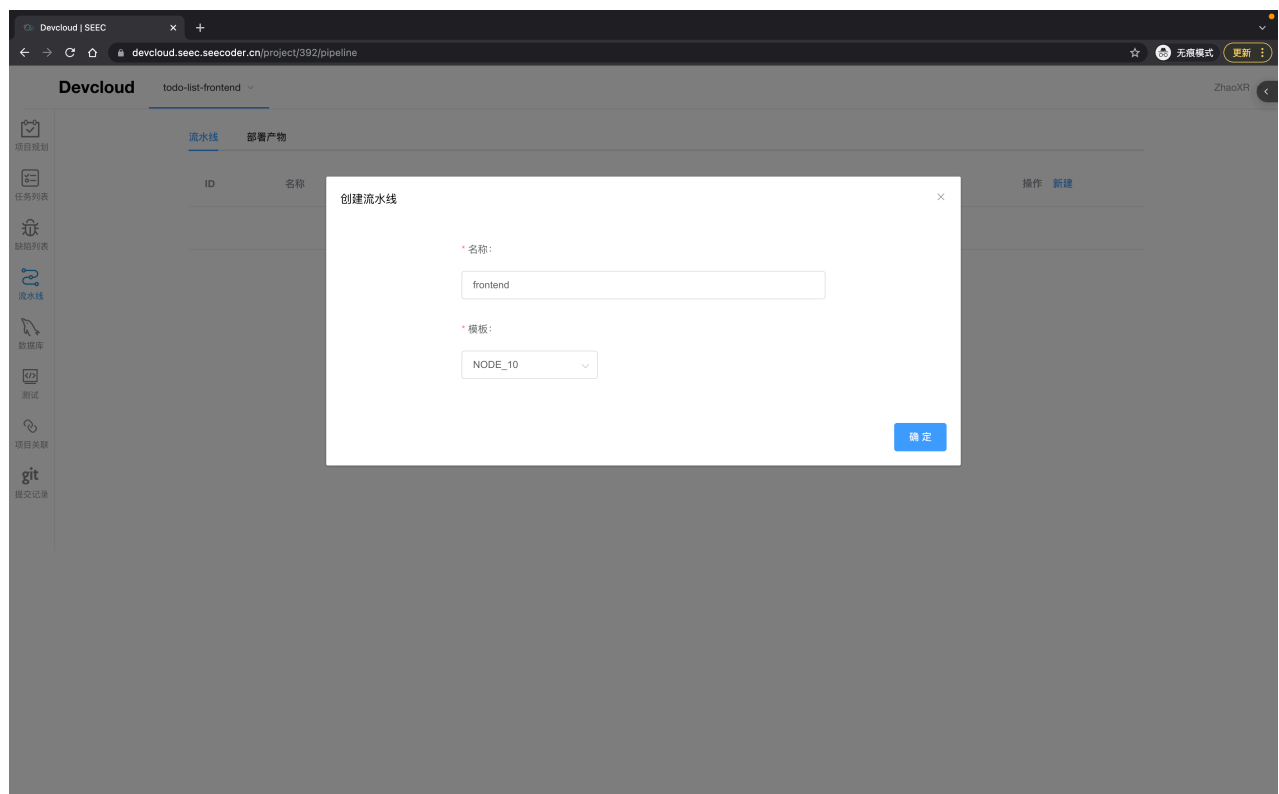


4.4 创建Vue.js前端实例

切换到前端项目

点击新建流水线

输入流水线名称，选择NODE_10模板



4.4.1 流水线配置

4.4.1.1 Node.js应用构建

1. repoUrl: 项目地址url用于clone, 不需要用户、密码验证信息, 请在结尾加上.git
2. branchName: 需要用于部署的分支, 请确保这条分支的应用可以正确运行, 默认是master
3. nginxConfig: 使用下面的配置, 注意填入的后端url需要稍作修改

- 后端url修改规则如下

我们在后端部署产物的界面, 看见的后端url为这种形式

backend.todo-list-backend-391.devcloud.seec.seecoder.cn

改为这种形式:

backend-service.todo-list-backend-391.svc.cluster.local

也就是在第一个分隔'.'前加入-service, 将后缀替换为.svc.cluster.local 将其填入配置

```
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
    root /var/www/html/dist;  
    location ^~/api/ {  
        proxy_pass http://backend-service.todo-list-backend-  
391.svc.cluster.local:8080/;  
        proxy_redirect http://backend-service.todo-list-backend-  
391.svc.cluster.local:8080/ /;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_set_header X-Forwarded-Port $server_port;  
    }  
    location ~ / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

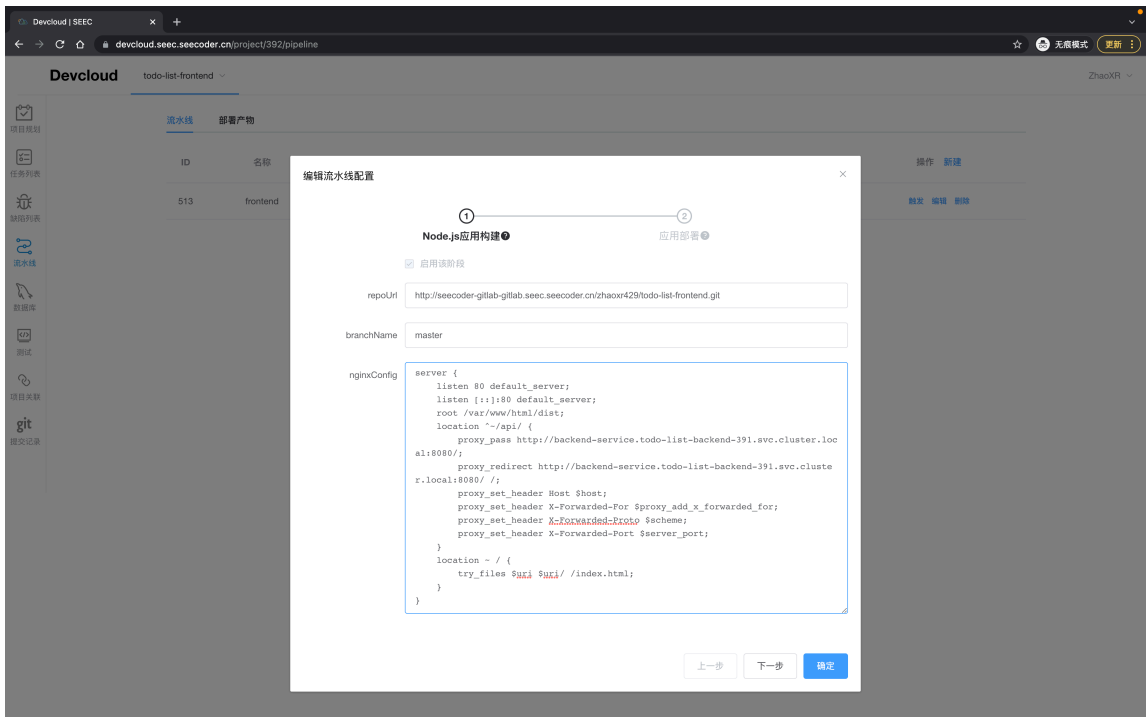
*.devcloud.seec.seecoder.cn 用于外部访问集群服务, 也就是上文可以使用postman对后端进行

接口调用的原因

*.svc.cluster.local 用于k8s集群内部访问自己的服务, 故这里要改为这种形式, 用于集群内部前端

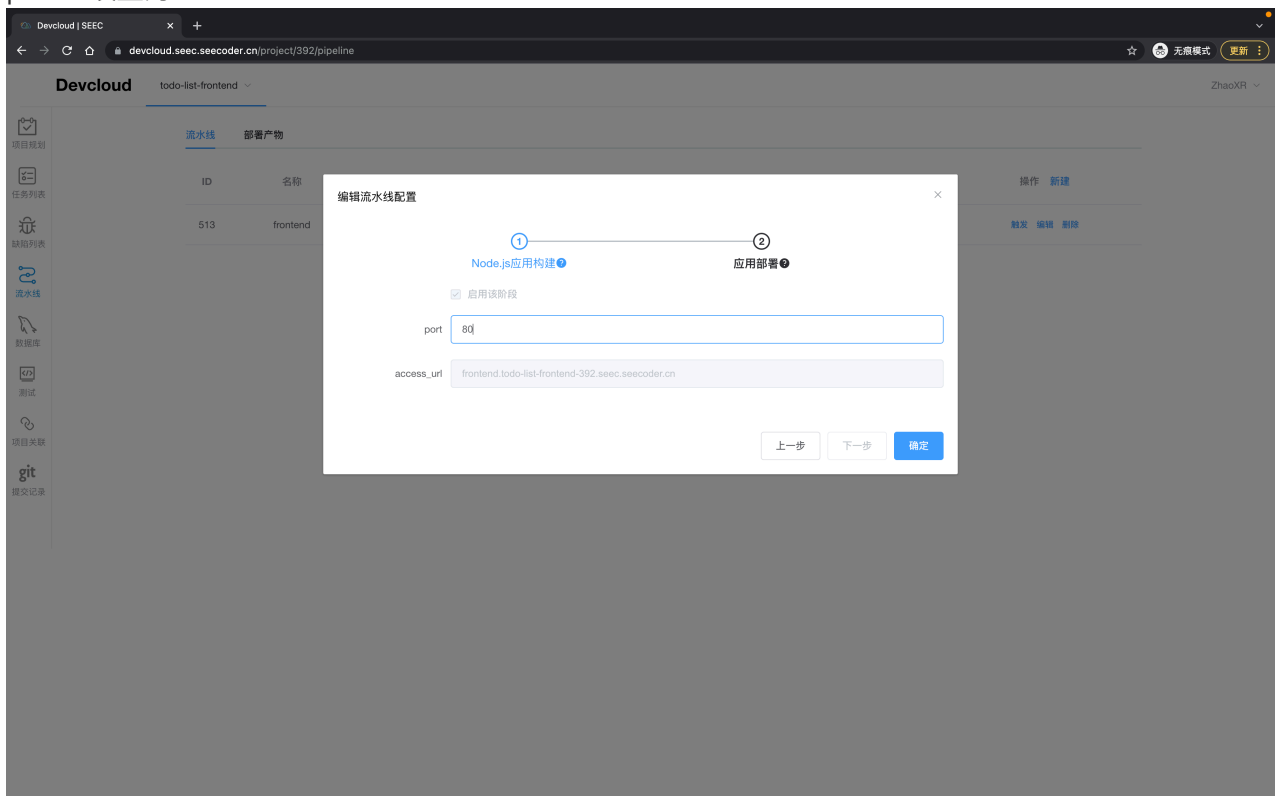
将请求转发给后端

这里仅作了解, 无需深入



4.4.1.2 应用部署

port: 设置为80



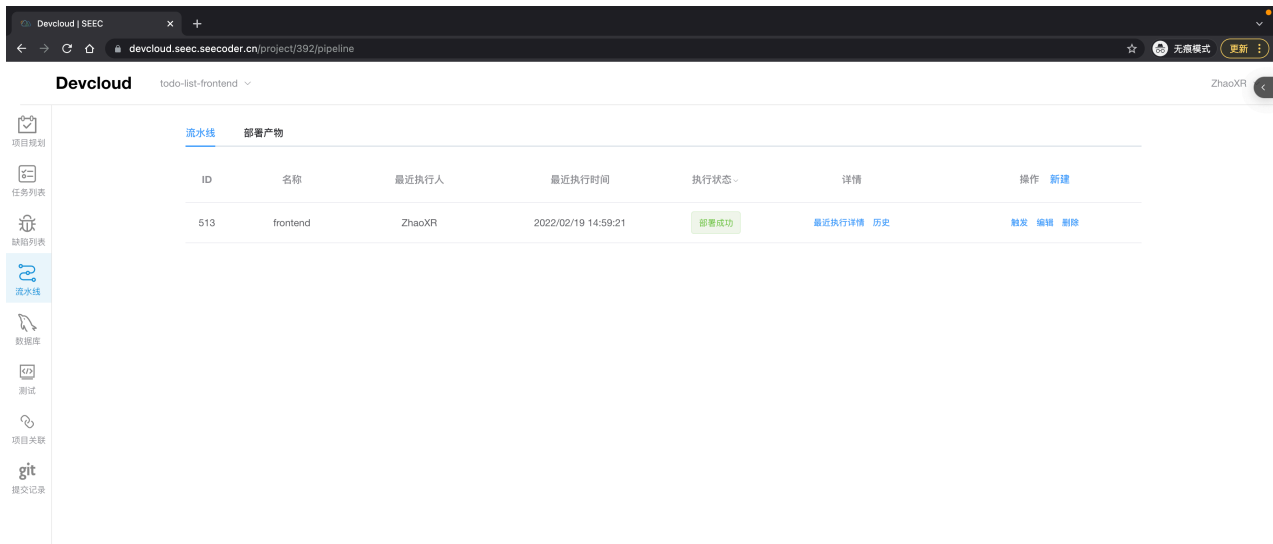
4.4.2 部署前端实例

点击触发进行部署

部署过程大概持续5-10min，请耐心等待

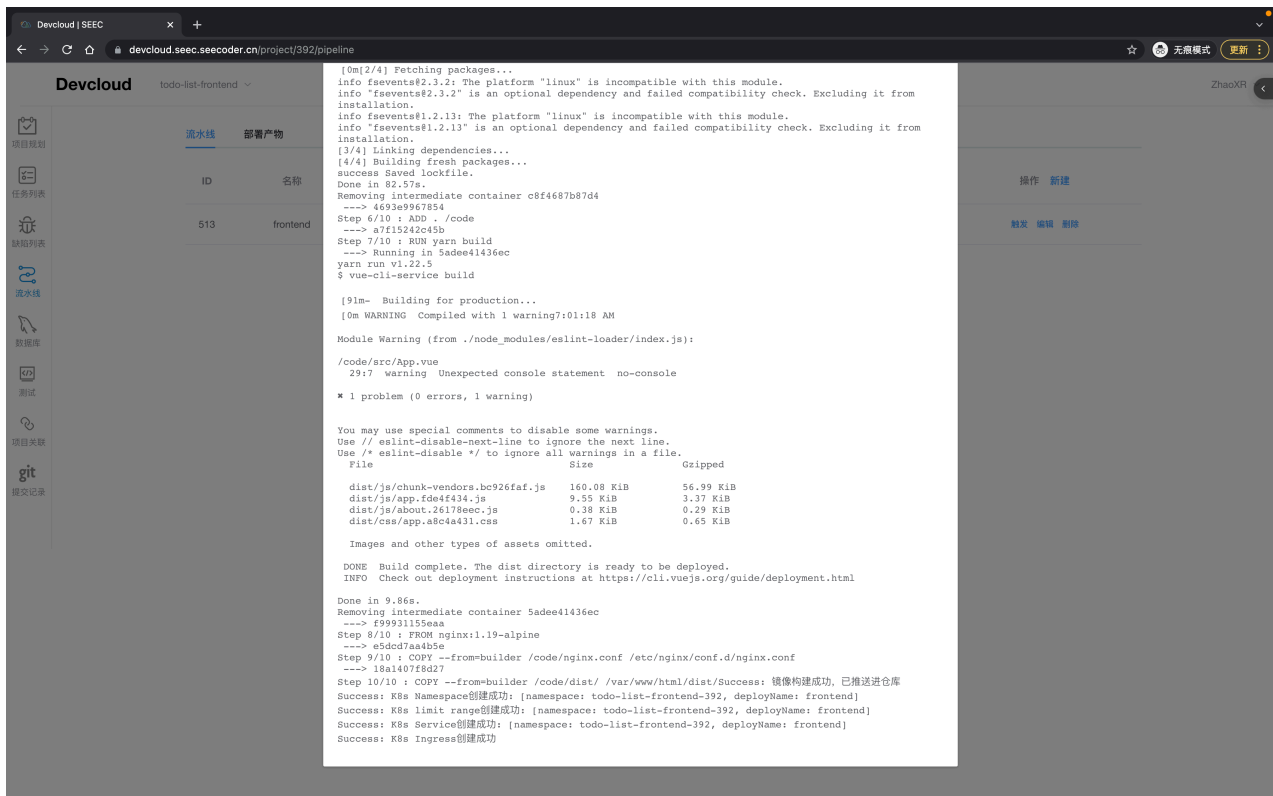
部署成功之前，部署产物会显示不存在（因为读取实例状态未读取到），请不要惊慌

部署成功如下：



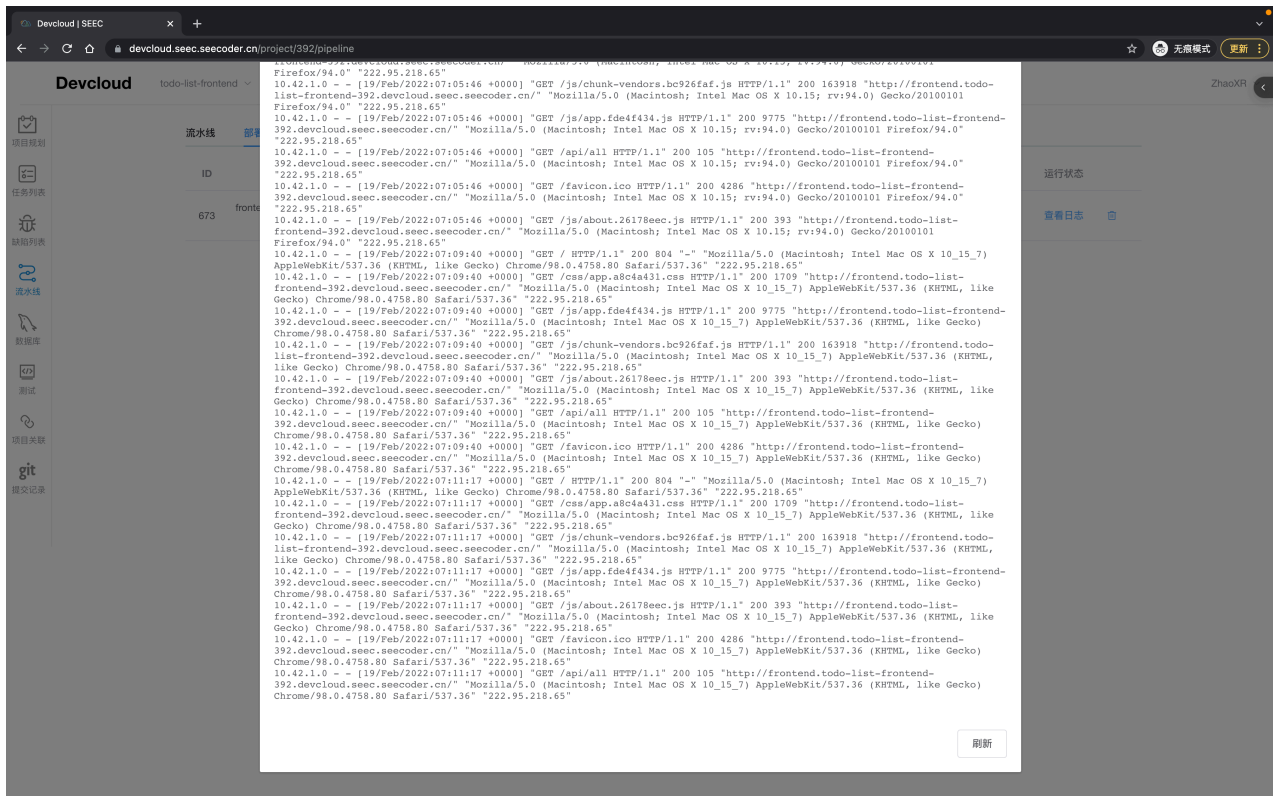
部署成功之后点击最后执行详情可以看见构建部署过程

部署失败请到也请到这里看部署日志，如果出现不能解决的问题联系我们



部署实例产物界面，并可以查看实例日志

前端线上Debug主要在浏览器Debug，这个实例日志没什么用的

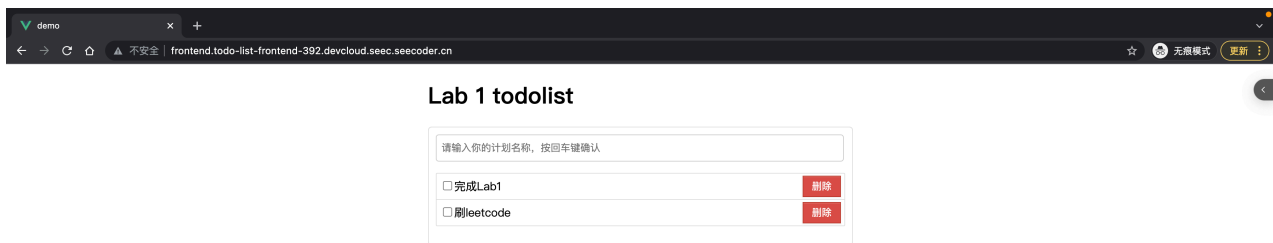


4.4.3 测试

在前端部署产物中复制url: <http://frontend.todo-list-frontend-392.devcloud.seec.seecoder.cn>

使用Chrome, http+无痕模式或者firefox

在浏览器中查看：



5. 测试功能

接口测试：为项目后端接口进行并发性能测试

黑盒测试：为项目后端的一些接口、函数、方法进行人工黑盒测试

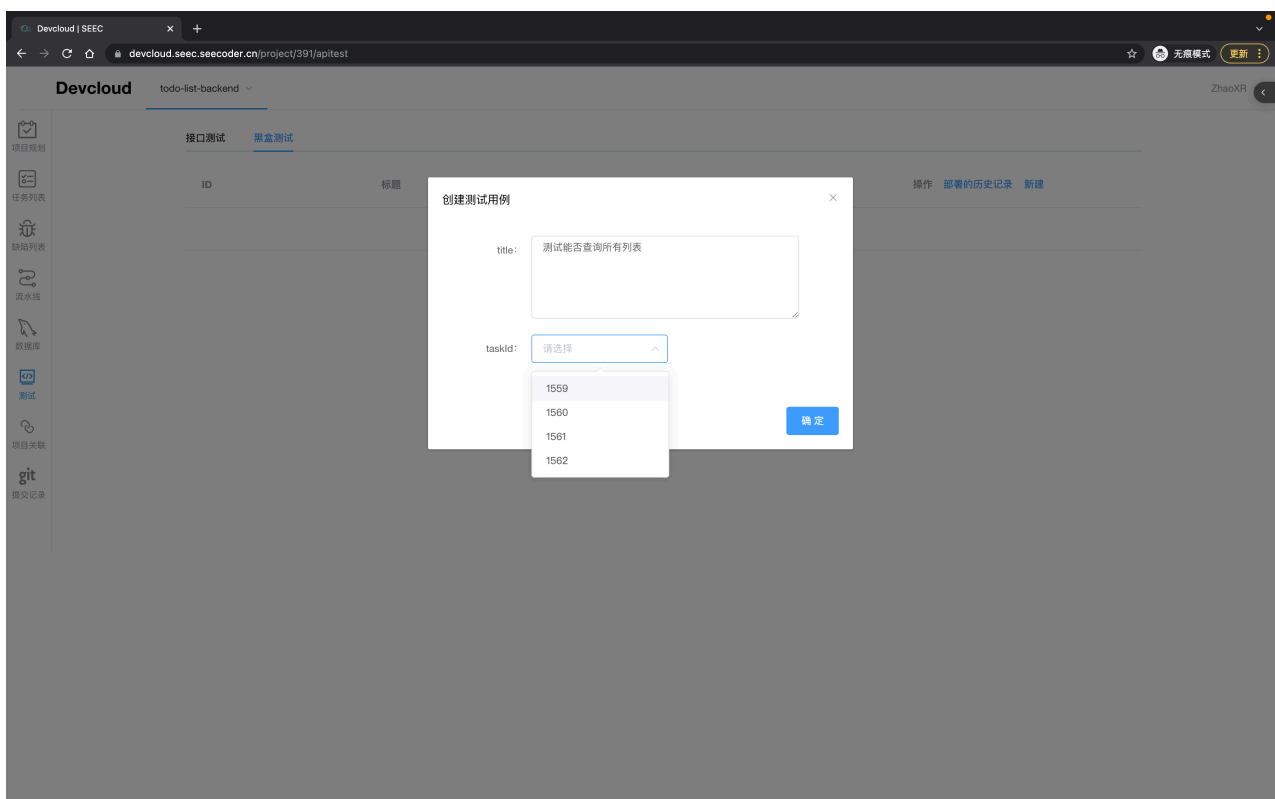
注：接口测试功能见Devcloud使用说明，本次实验不使用接口测试功能，测试接口请使用postman等工具

5.1 黑盒测试

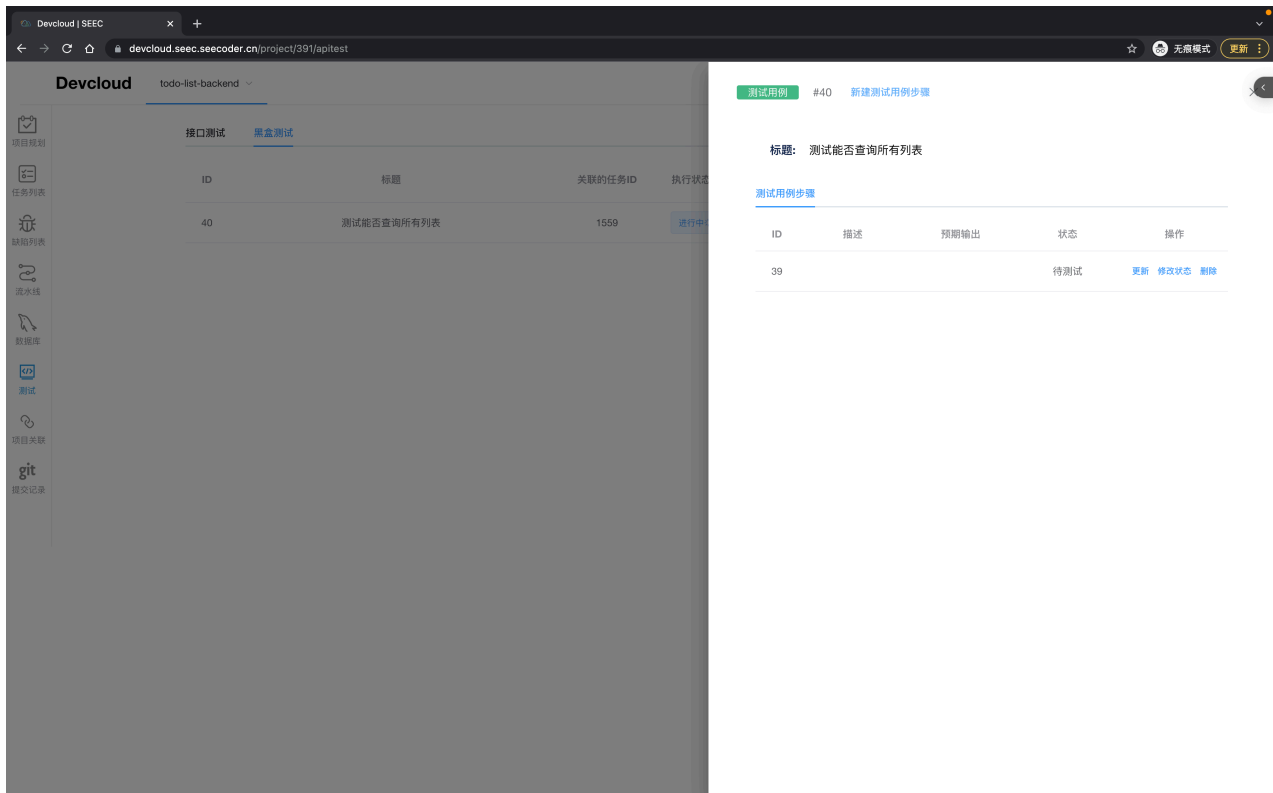
手动验证后端接口的预期输出和实际输出对比

切换到后端项目，点击导航栏第六项 **测试** 下的黑盒测试Tab

点击创建，填入title并选择taskId（这里的taskId对应任务列表中的id）

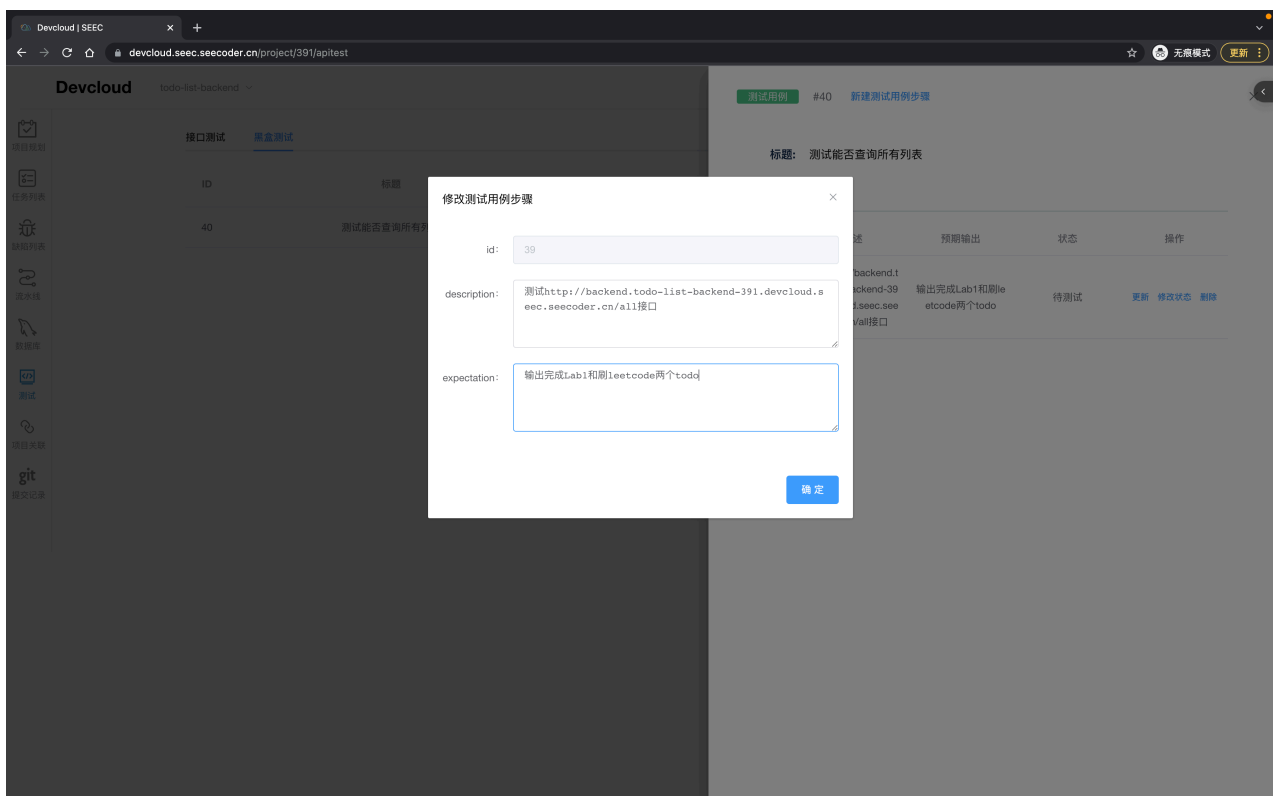


点击测试用例步骤列表，可以查看属于该测试用例的步骤列表，点击新建测试用例步骤后生成一个空的步骤

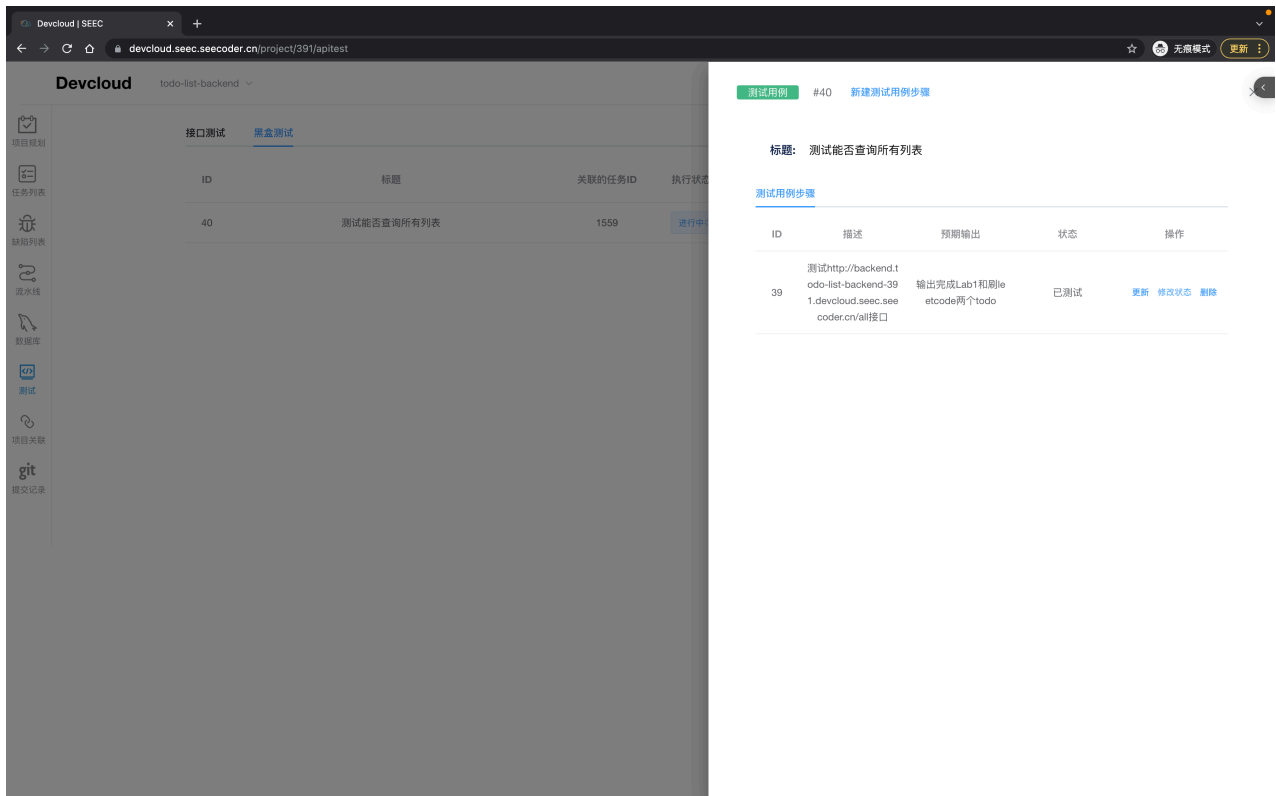


测试用例步骤有描述、预期输出和状态三个属性，可以更新步骤的描述和预期输出以及更新状态

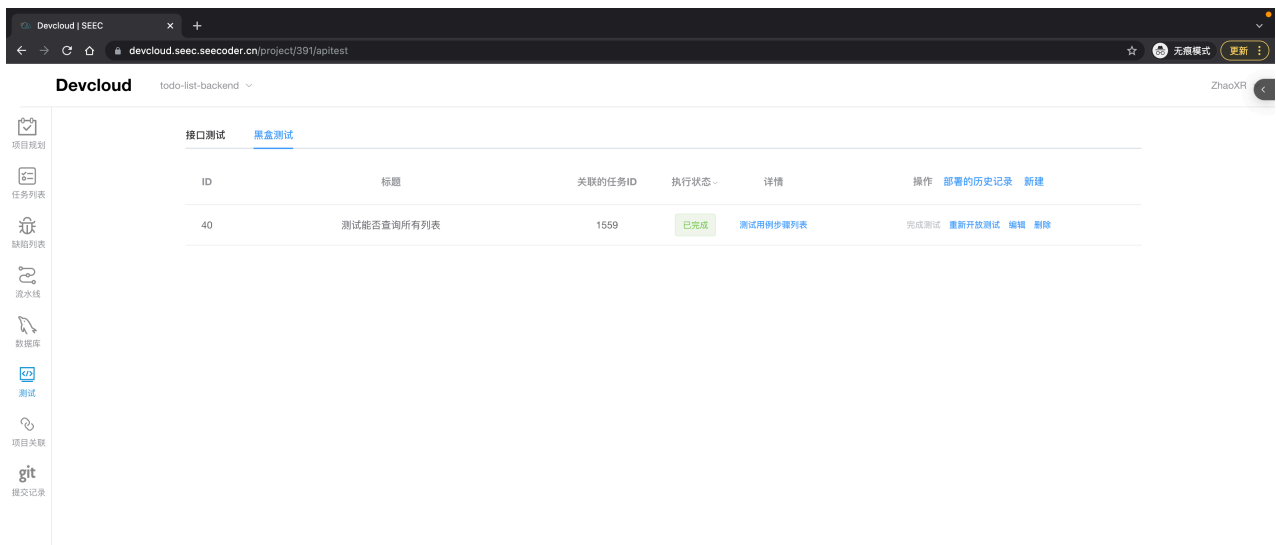
下图是更新测试用例步骤



下图是手动完成后，将测试用例状态修改为已测试



所有测试用例步骤通过后，可以点击完成测试，将测试用例的状态修改为已完成（对测试用例有疑问，也可以重新开放测试用例）



请各位同学自行设计黑盒测试

6. 项目关联功能

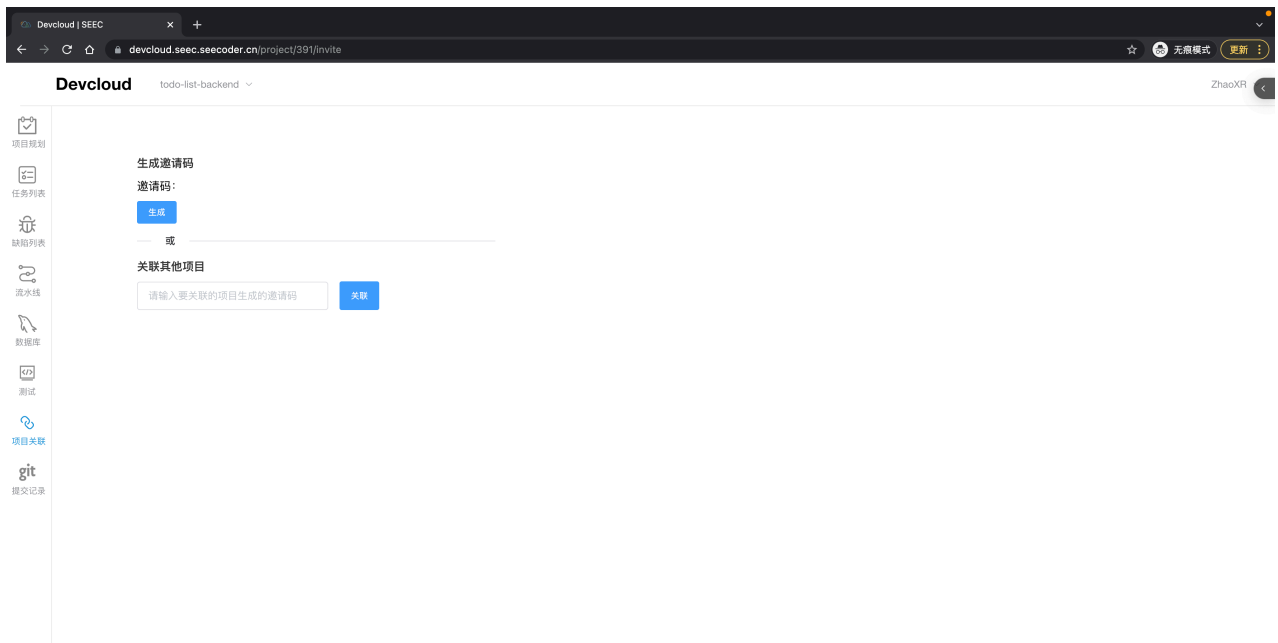
点击导航栏第七项项目关联

可以生成邀请码，其他项目填写邀请码，将两个项目关联起来

or

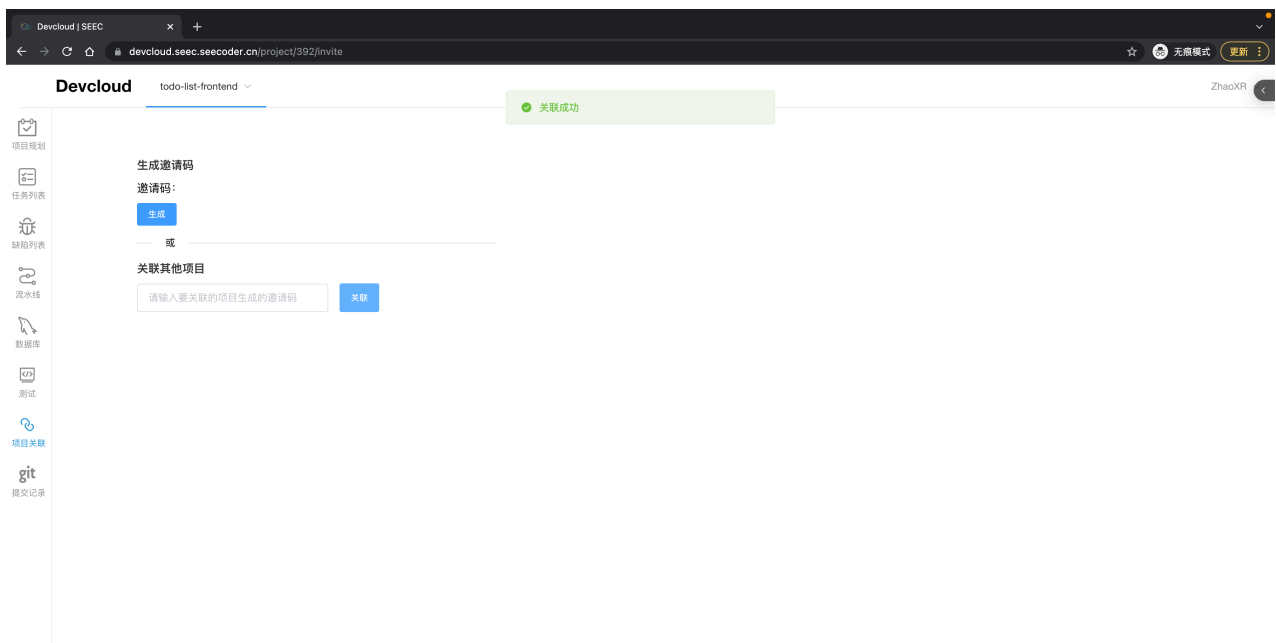
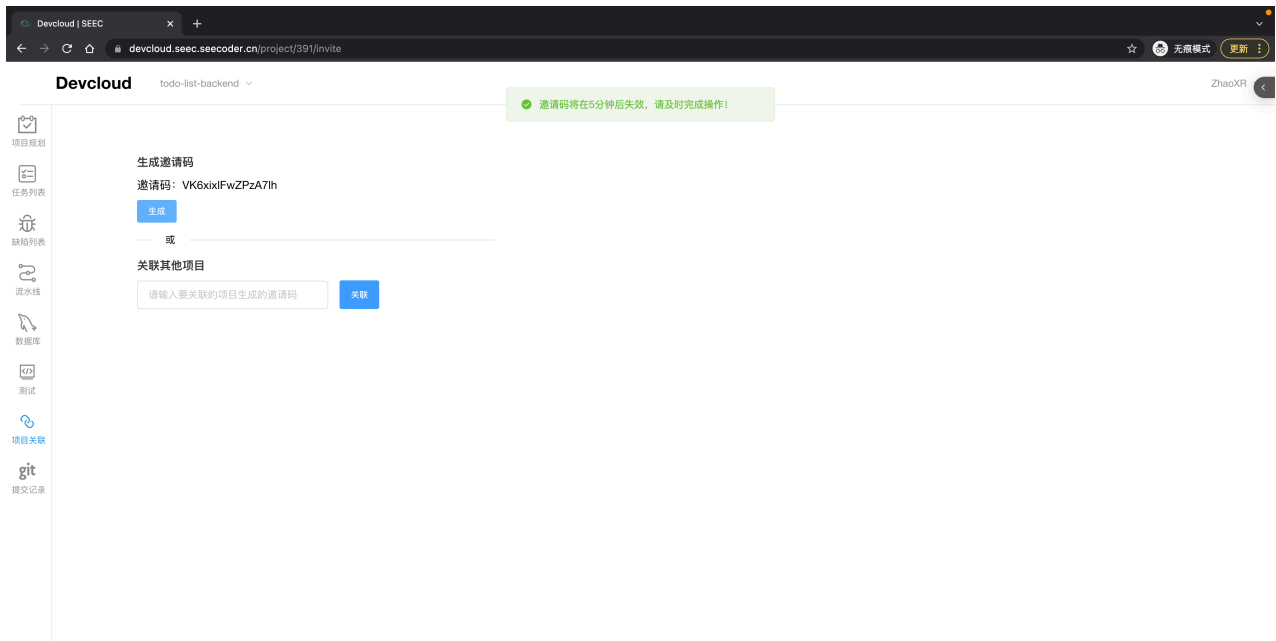
填写其他项目的邀请码

这里可以将前后端项目关联起来



邀请码的有效时间是5分钟，请及时填写

目前关联的项目缺陷记录是可以共享的



7. 查看git记录功能

点击导航栏第八项git记录，可以查看该项目与task和bug操作的git记录
修改task和bug的状态不局限于通过git操作实现，手动修改状态也可以

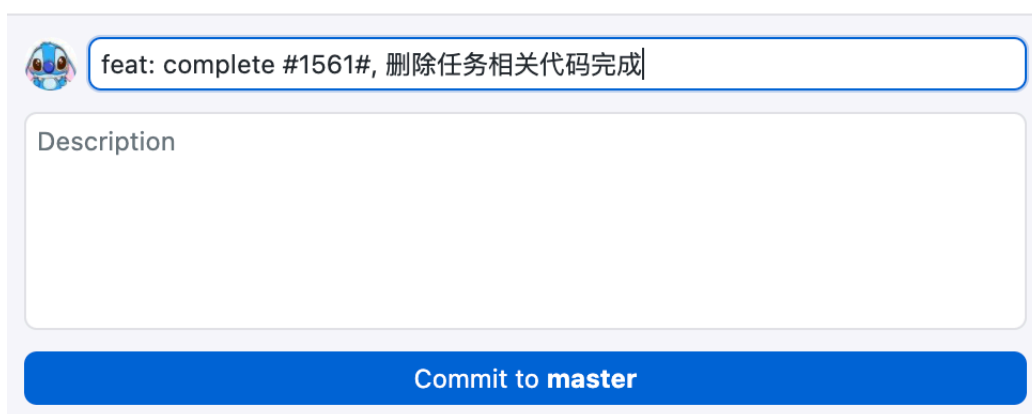
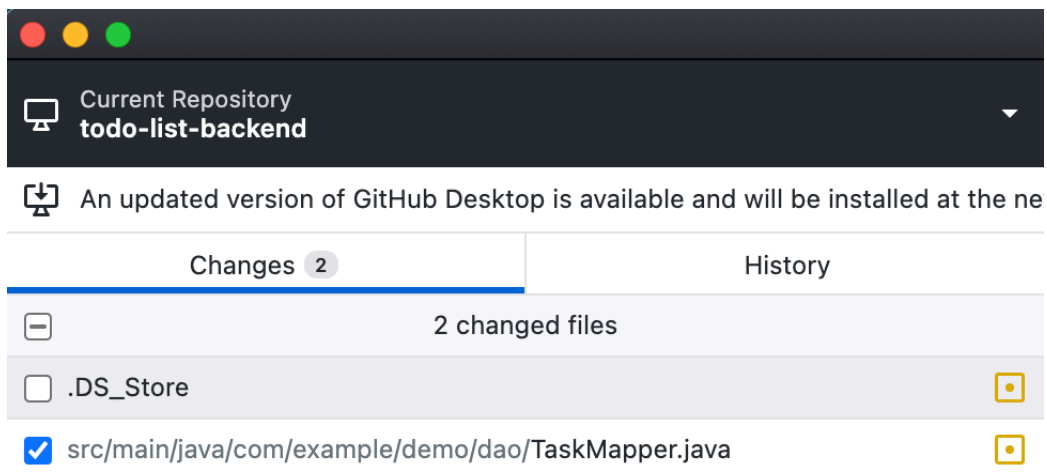
Task

使用如下代码提交规范：

```
feat: working on #1560#, 添加新任务service完成
feat: complete #1561#, 删除任务相关代码完成
```

前者会将id为1560的Task的状态变为正在进行

后者会将id为1561的Task的状态变为已完成



提交前的task列表如下：

Devcloud | SEEC

devcloud.seec.seecoder.cn/project/391/taskList

无痕模式更新

Devcloud

todo-list-backend

ZhaoXR

项目规划

任务列表

缺陷列表

流水线

数据库

测试

项目关联

git

提交记录

清除过滤器

ID	所属项目ID	优先级	名称	负责人	预期耗时(人日)	开始时间	结束时间	状态
1559	391	宽松	获取所有任务	ZhaoXR	1	2022/02/19 13:02:54	2022/02/19 13:04:59	已结束
1560	391	宽松	添加新任务	TestUser	1	2022/02/19 13:02:55		已创建
1561	391	紧急	删除任务	ZhaoXR	1	2022/02/19 13:02:57		进行中
1562	391	一般	更新任务	TestUser	1	2022/02/19 13:02:58		进行中

提交后的task列表及git记录如下：

Devcloud | SEEC

devcloud.seec.seecoder.cn/project/391/taskList

无痕模式更新

Devcloud

todo-list-backend

ZhaoXR

项目规划

任务列表

缺陷列表

流水线

数据库

测试

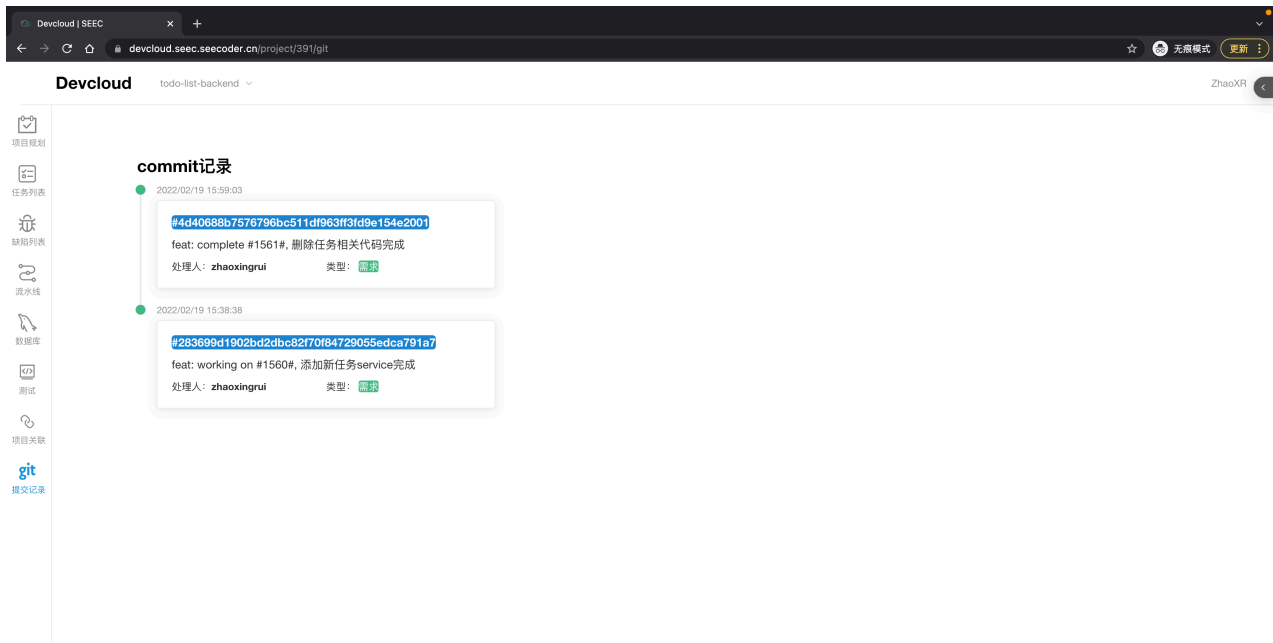
项目关联

git

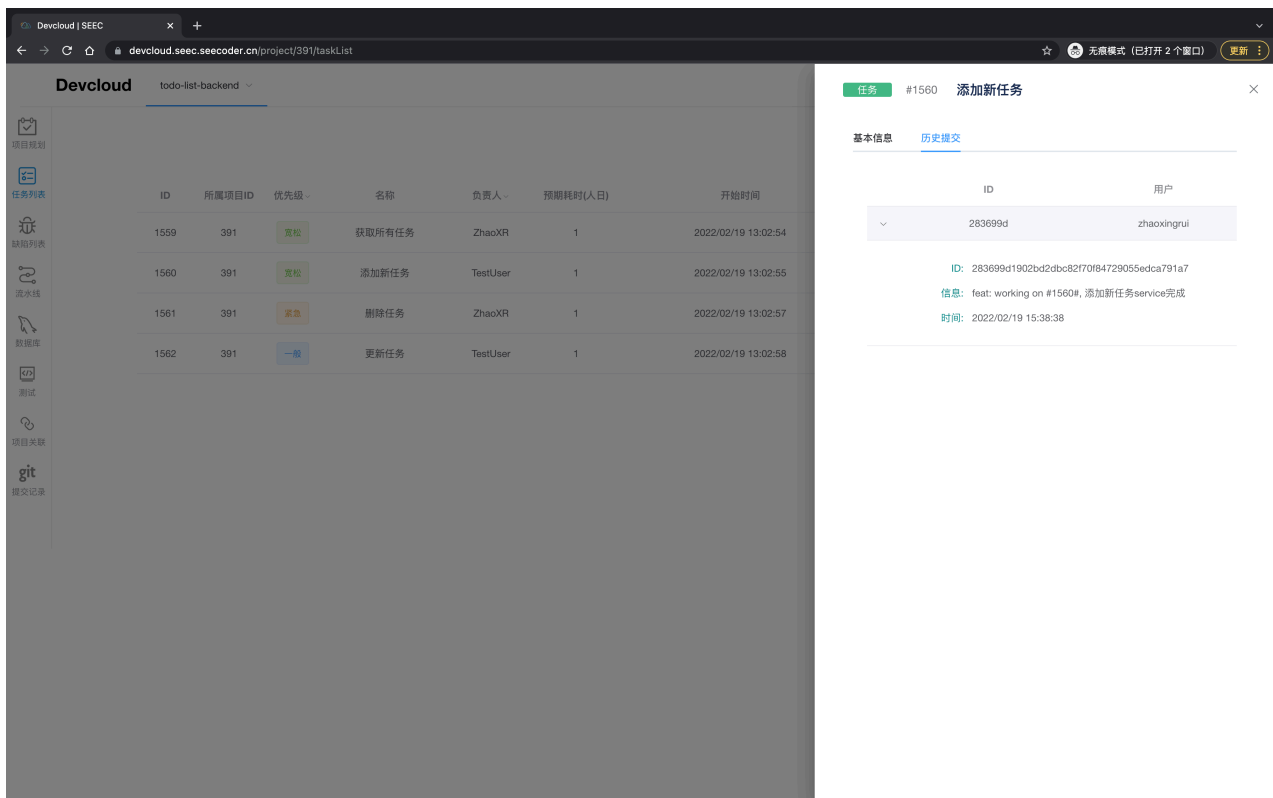
提交记录

清除过滤器

ID	所属项目ID	优先级	名称	负责人	预期耗时(人日)	开始时间	结束时间	状态
1559	391	宽松	获取所有任务	ZhaoXR	1	2022/02/19 13:02:54	2022/02/19 13:04:59	已结束
1560	391	宽松	添加新任务	TestUser	1	2022/02/19 13:02:55		进行中
1561	391	紧急	删除任务	ZhaoXR	1	2022/02/19 13:02:57	2022/02/19 15:59:05	已结束
1562	391	一般	更新任务	TestUser	1	2022/02/19 13:02:58		进行中



同时在task列表中的每一项也可以查看提交记录



bug

使用以下代码提交规范：

```
fix: fixing on #xx#, 修改xxxx
fix: fix #91#, 提交空title的bug已解决
```

前者会将id为xx的bug的状态变为进行中

后者会将id为91的bug的状态变为已完成

提交前的bug列表如下：

Devcloud | SEEC

devcloud.seec.seecoder.cn/project/392/bugList

无虞模式 (已打开 2 个窗口)更新

Devcloud

todo-list-frontendZhaoXR

项目规划

任务列表

缺陷列表

流水线

数据库

测试

项目关联

提交记录

清除过滤器新建

ID	所属项目ID	优先级	名称	负责人	预期耗时(人日)	开始时间	结束时间	状态
90	392	紧急	点击更改任务状态无效	ZhaoXR	1	2022/02/19 13:19:34		已创建
91	392	一般	增加新任务的title为空时不报错	ZhaoXR	1	2022/02/19 13:20:15		已创建

提交后的bug列表及commit记录如下：

Devcloud | SEEC

devcloud.seec.seecoder.cn/project/392/bugList

无虞模式更新

Devcloud

todo-list-frontendZhaoXR

项目规划

任务列表

缺陷列表

流水线

数据库

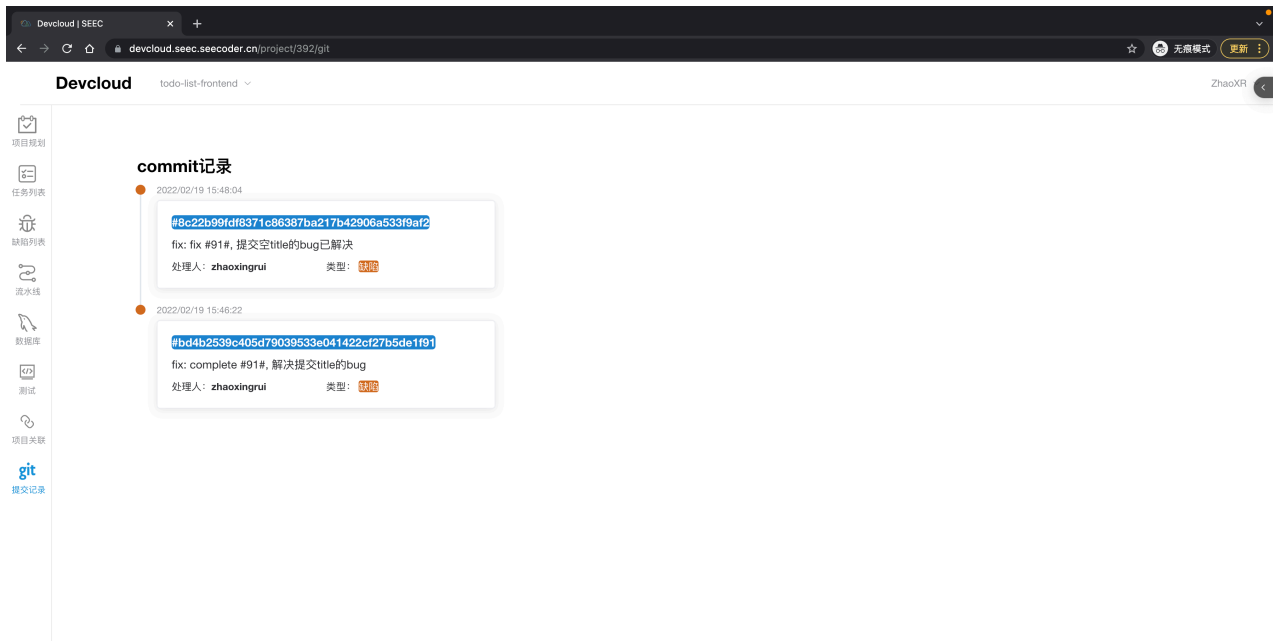
测试

项目关联

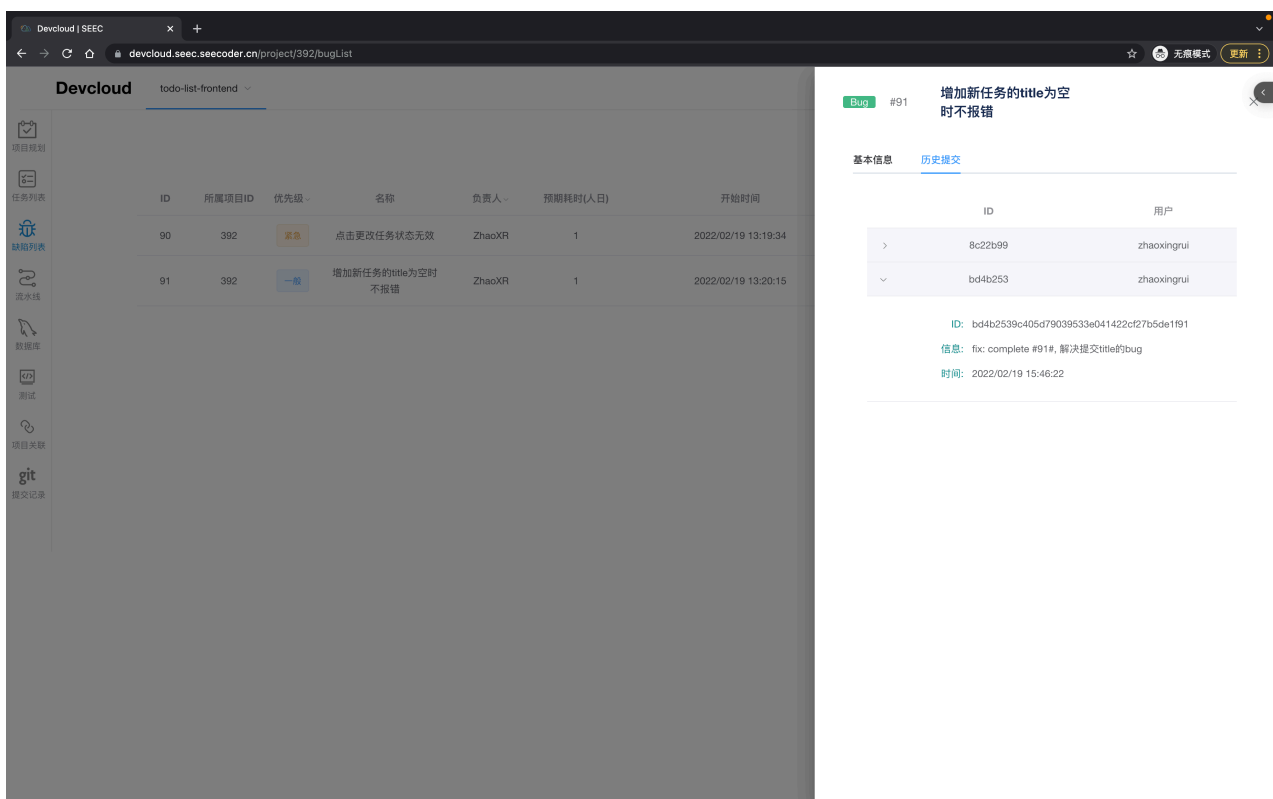
提交记录

清除过滤器新建

ID	所属项目ID	优先级	名称	负责人	预期耗时(人日)	开始时间	结束时间	状态
90	392	紧急	点击更改任务状态无效	ZhaoXR	1	2022/02/19 13:19:34		已创建
91	392	一般	增加新任务的title为空时不报错	ZhaoXR	1	2022/02/19 13:20:15	2022/02/19 15:48:07	已结束



同时在bug列表中的每一项也可以查看提交记录



二. Lab1作业要求

1. 作业要求

1. 按照Lab1流程完成前后端代码的部署
2. 自行构建需求规划树；找到项目中的两个bug，在缺陷列表中记录，并在黑盒测试中记录
3. 解决项目中的两个bug
 1. 示例代码中无法更新任务状态，请解决，对应代码在后端TaskServiceImpl.java文件中
 2. 示例代码允许提交空title的todo，请修改成如果提交空title，浏览器提示“title不可为空”，对应代码在前端Header.vue中

下图是允许提交空title的情况

Lab 1 todolist



请输入你的计划名称，按回车键确认	
☐ 完成Lab1	删除
☐ 刷leetcode	删除
☐	删除

4. 在前端代码中添加成员信息表格，用户名，学号即可（在Lab 1 todolist标题下方添加即可）

2. 作业提交

1. 提交pdf文件，在pdf文件中标注小组成员姓名及学号，同时标注分工（需求书写，缺陷书写，数据库、表、数据初始化，前后端流水线及部署，bug解决等），文件名组长学号_组长姓名，只需要组长提交即可
2. 在pdf文件中写出前后端访问url
3. 在pdf文件中给出项目规划中的需求树截图，任务列表截图，缺陷列表截图，前后端流水线部署成功日志及部署产物日志，数据库、表、数据初始化执行结果截图，黑盒测试截图，bug解决方案（给出相关文件的代码块）
4. 提交时间及方式后续另行通知